$P = NP$ by Exclusion of the SAT Separator Endpoint

An AASC Endpoint-Rigidity Proof for Arbitrary Finite CNF-SAT

Amos Jay Maley

June 2026

Abstract

This manuscript proves $P = NP$ by excluding the SAT separator endpoint on the unrestricted finite CNF-SAT carrier. The proof is AASC-owned: AASC supplies the endpoint-rigidity argument that excludes the bare negative branch, while Cook–Levin/Karp supplies only the standard endpoint correspondence from polynomial-time CNF-SAT to $P = NP$. The positive raw branch is $\mathsf{Pos}_{\mathsf{raw}} := \mathsf{CnfSATInPolyTime}$, asserting deterministic polynomial-time decidability of arbitrary finite CNF-SAT. The bare negative branch $\mathsf{Sep}_{\mathsf{bare}}$ is the raw complement of $\mathsf{Pos}_{\mathsf{raw}}$ on the same carrier.

The proof is kernel-first. Theorem-bearing endpoint use is treated as governed construction, and governed construction is possible only under the kernel of admissibility, standing, reference, and irreversibility. The downstream AASC machinery is therefore not an external classifier or a source of theorem standing; it is the fixed-domain consequence layer forced by kernel necessity. Ordinary theoremhood, positive or negative, is kernel-internal when it is genuine theoremhood rather than raw trace.

The decisive step is the SAT-local endpoint asymmetry. The positive branch directly occupies the constructive endpoint carrier. The raw negative branch, by contrast, is transported by the SAT lower-bound/no-positive equivalence into the SAT image-separator branch. That branch induces an independent same-domain endpoint-status discriminator on the fixed CNF-SAT carrier. Kernel-forced structural rigidity excludes such discriminators. Hence the SAT separator route is unavailable.

It follows that $\mathsf{Sep}_{\mathsf{bare}}$ is excluded. By raw complementarity, $\mathsf{Sep}_{\mathsf{bare}} \leftrightarrow \neg\mathsf{Pos}_{\mathsf{raw}}$, the positive branch follows. Therefore arbitrary finite CNF-SAT is decidable in deterministic polynomial time. The final passage from $\mathsf{CnfSATInPolyTime}$ to $P = NP$ is the standard Cook–Levin/Karp endpoint correspondence. The proof of branch exclusion is AASC-internal; external complexity theory is used only to identify the official endpoint consequence.

Dependency-Inversion Exclusion

A recurring possible misreading must be excluded at the outset. The manuscript does not argue from an optional AASC framework to a special proof standard and then apply that standard to SAT. The dependency order is the reverse. The demand that an alleged proof, counterproof, endpoint resolution, or theorem-bearing act be determinate already requires a fixed target, evaluable steps, act-time finality, and same-regime fidelity. These four requirements force the roles later named reference, standing, irreversibility, and admissibility. AASC is the systematic consequence layer of those forced roles.

Thus a reading that treats the kernel as an optional framework premise is a dependency inversion. It changes the argument from a necessity proof into a framework-application proof. That is not a

competing interpretation of the manuscript. It is a denial of the kernel-neutral adequacy argument and must be assessed only by identifying a specific failure in the passage from target adequacy to the kernel roles.

Contents

1 Proof Class, Endpoint Validity, and Kernel-First Discipline

1.1 Proof-class lock

The proof class is

kernel-forced endpoint-image discriminator closure.

It is not a circuit lower bound, diagonalization argument, relativization argument, natural-proofs argument, algebrization argument, proof-complexity lower bound, exhaustive-search lower bound, or empirical solver-hardness claim. It is a main-text mathematical proof about which theorem-bearing endpoint statuses can survive on the fixed SAT endpoint image once the kernel of non-degenerate construction is in force.

1.2 Necessity, not optional framework

The argument does not introduce AASC as an optional foundational system layered on top of the P versus NP problem. The kernel of non-degenerate construction, and the downstream consequences used in the proof, are derived as necessities forced by the minimal evaluability conditions required for any raw sequence to count as construction of a determinate same-domain object. These conditions are stated in kernel-neutral form before governed construction is defined. Once they are fixed, the governance roles of admissibility, standing, reference, and irreversibility follow as forced roles; the fixed-domain consequences of those roles include the structural rigidity principles that exclude independent same-domain endpoint-status discriminators. AASC is the systematic development of those necessity constraints. The positive endpoint is therefore not obtained by adopting a proprietary framework, but by applying the consequences of non-degenerate construction to the official unrestricted finite CNF-SAT carrier.

1.3 Clay-valid as official mathematical endpoint

Definition 1.1 (Clay-valid mathematical endpoint)**.** In this manuscript, "Clay-valid" is used only in the mathematical endpoint sense. A Clay-valid endpoint targets the official P versus NP problem, preserves the standard definitions of P, NP, polynomial time, SAT, CNF-SAT, polynomial-time reductions, and NP-completeness, and reaches one of the official mathematical branches. The term is used only for this mathematical endpoint property.

1.4 Endpoint and method

The branch-exclusion step is supplied by the necessity constraints already forced by kernel-neutral target adequacy on the official carrier. AASC records those constraints and their fixed-domain consequences; it does not supply an optional foundational layer. The proof is not a diagonalization argument, circuit lower-bound argument, relativization argument, natural-proofs argument, proof-complexity argument, or constructive algorithm search. Classical complexity theory enters only at the endpoint-correspondence layer: unrestricted finite CNF-SAT is the standard NP-complete carrier, and deterministic polynomial-time decidability of that carrier entails $\mathsf{P} = \mathsf{NP}$ by the Cook–Levin/Karp correspondence. The proof of $\neg\mathsf{Sep}_{\mathsf{bare}}$ is AASC-internal.

1.5 Kernel-first ordering

The central dependency is

$$\mathsf{EndpointUse}_R(B) \Rightarrow \text{governed construction} \Rightarrow K_R \Rightarrow \mathsf{A}_R^{+}(B).$$

The reverse dependency is not used. AASC does not create standing, and reportability does not supply standing. Standing is one member of the kernel already presupposed by genuine governed construction. A proof of a proposition, when it is a proof rather than raw syntax, is therefore already kernel-internal.

Proposition 1.2 (Downstream status of AASC machinery)**.** *In this manuscript, AASC, UEAP, ATS, AMetricity, no-selector discipline, no-third-status discipline, unique-interior discipline, standing conservation, and report-preservation discipline are downstream consequence machinery forced by the kernel under fixed-domain theorem-bearing endpoint use. None of them is a primitive source of theorem standing.*

Proof. The kernel proof in Section 3 begins with raw step-sequences and kernel-neutral target adequacy, then proves that target determinacy, step evaluability, act-time finality, and same-regime fidelity force reference, standing, irreversibility, and admissibility. The later AASC machinery is the endpoint interface of those roles: it preserves target, carrier, admissibility boundary, theorem-status reuse, redescription stability, and report integrity. Therefore it records forced consequences of the kernel rather than adding an independent source of standing. □

1.6 The SAT context object is neutral

Definition 1.3 (Fixed SAT endpoint context)**.** The context object

$$\mathcal{C}_{\mathsf{SAT}}(R, m)$$

records only the official SAT/P-vs-NP endpoint setting: the unrestricted finite CNF-SAT carrier, the raw positive branch $\mathsf{Pos}_{\mathsf{raw}}$, the raw negative branch $\mathsf{Sep}_{\mathsf{bare}}$, the SAT image-separator branch $\mathsf{Sep}_{\mathsf{img}}$, and the fixed model/carrier on which the raw branches are evaluated. It is a carrier-and-slot object. It is not an A+ certificate, not an AMetric premise, not a no-selector axiom, not a separator-classification premise, and not a positive endpoint premise.

Theorem 1.4 (Context non-smuggling)**.** *The context object* $\mathcal{C}_{\mathsf{SAT}}(R, m)$ *does not assert any of*

$$\mathsf{A}^{+}_{R}(B), \quad \mathsf{AMetricBoundary}(R), \quad \mathsf{A}_{\mathsf{sep}}, \quad \neg\mathsf{Sep}_{\mathsf{bare}}, \quad \neg\mathsf{Sep}_{\mathsf{img}}, \quad \mathsf{Pos}_{\mathsf{raw}}.$$

It fixes the official SAT carrier and branch slots only.

Proof. By Definition 1.3, the clauses of $\mathcal{C}_{\mathsf{SAT}}(R, m)$ are carrier and slot clauses. They say which endpoint image is under evaluation and which raw branches are being evaluated on that image. They do not classify any branch, exclude any branch, or assert the positive branch. All endpoint-status consequences are derived later from endpoint use plus the kernel. □

1.7 Endpoint layers

Definition 1.5 (Bare proposition layer)**.** The raw positive proposition is

$$\mathsf{Pos}_{\mathsf{raw}} := \mathsf{CnfSATInPolyTime}.$$

The bare negative proposition $\mathsf{Sep}_{\mathsf{bare}}$ is the raw complement branch for the same unrestricted finite CNF-SAT carrier.

Definition 1.6 (SAT image-separator branch)**.** The SAT image-separator branch $\mathsf{Sep}_{\mathsf{img}}$ denotes the exact manuscript counterpart of the formal object $\mathsf{CnfSATImageSeparatorBranch}(R, model)$. It is the SAT lower-bound/no-positive branch after transport through the fixed SAT operator bridge. The manuscript does not insert an additional residual layer between the raw negative branch and $\mathsf{Sep}_{\mathsf{img}}$: the formal bridge used below is exactly

$$\mathsf{CnfSATBareNegativeBranch}(R, model) \Rightarrow \mathsf{CnfSATImageSeparatorBranch}(R, model).$$

Thus $\mathsf{Sep}_{\mathsf{img}}$ is not a generic report act, not ordinary negative theoremhood by itself, and not an A+ premise. It is the image-separator branch used by the SAT-local discriminator theorem.

Definition 1.7 (Endpoint classification layer)**.** The symbol $\mathsf{A}_{\mathsf{sep}}$ denotes the role-specific separator endpoint-status factor, if such a factor is asserted for endpoint-image separator occupation $\mathsf{Sep}_{\mathsf{img}}$. The Clay-valid separator endpoint is

$$\mathsf{Sep}_{\mathsf{Clay}} := \mathsf{Sep}_{\mathsf{img}} \wedge \mathsf{A}_{\mathsf{sep}}.$$

The positive report classification is $\mathsf{A}_{\mathsf{pos}}$, and the Clay-valid positive endpoint is

$$\mathsf{Pos}_{\mathsf{Clay}} := \mathsf{Pos}_{\mathsf{raw}} \wedge \mathsf{A}_{\mathsf{pos}}.$$

These classification symbols are downstream status factors; they do not create kernel standing.

2 Official Target and SAT Endpoint Carrier

2.1 The official P versus NP target

The official P versus NP problem asks whether every language accepted by a nondeterministic polynomial-time algorithm is also accepted by a deterministic polynomial-time algorithm. The problem statement is

$$\mathsf{P} \stackrel{?}{=} \mathsf{NP}.$$

The proof works through encoded CNF satisfiability as the SAT endpoint carrier.

2.2 CNF satisfiability

For a CNF formula φ, an assignment a assigns Boolean values to its variables, and

$$SAT(\varphi) \quad \Longleftrightarrow \quad \exists a \, Eval(\varphi, a) = 1.$$

The formal carrier uses finite encodings of CNF formulas, the Boolean characteristic function $\mathsf{satChar}$, and a Turing-machine polynomial-time certificate for the positive endpoint.

Definition 2.1 (Unrestricted finite CNF carrier; bounded-CNF clarification)**.** Throughout this manuscript, "bounded-CNF" means the ordinary finitely encoded CNF-SAT carrier: arbitrary finite CNF formulas over a finite Boolean alphabet, with input length equal to the size of the encoded formula. It does not mean fixed-width k-CNF, fixed clause count, fixed variable count, bounded instance size, bounded search depth, promise-restricted CNF, parameterized tractable fragments, or any other restricted SAT fragment. Thus

$$\mathsf{CnfSATInPolyTime}$$

is the assertion that arbitrary finite CNF-SAT is decidable by a deterministic polynomial-time machine.

2.3 Raw bivalence is bare-level bivalence

Raw bivalence is stated at the bare layer:

$$\mathsf{Pos}_{\mathsf{raw}} \vee \mathsf{Sep}_{\mathsf{bare}}.$$

The final proof does not infer reportability from the raw disjunction. It uses the SAT-local lower-bound/no-positive bridge

$$\mathsf{Sep}_{\mathsf{bare}} \Rightarrow \mathsf{Sep}_{\mathsf{img}}$$

to move the raw negative branch directly into the SAT image-separator branch. No reportability theorem is used in the bare-branch exclusion. Report acts reappear only after the positive endpoint has been derived.

3 Self-Contained Kernel Proof for Non-Degenerate Construction

This section internalizes the kernel proof in the manuscript. The kernel is not cited as an external AASC preference and is not treated as an appendix assumption. The definitions below begin with raw step-sequences and kernel-neutral target adequacy, prove that the roles of reference, standing, irreversibility, and admissibility are forced by determinate same-domain construction, prove non-derivability and no-faithful-lower-generator results, and derive the fixed-domain consequence package used later by the SAT endpoint argument.

3.1 Fixed-Domain Construction and Minimal Setting

This section records the minimal structural conditions under which a raw step-sequence is assessable as construction of a determinate object on a fixed domain. They are not introduced as optional definitions for a preferred formalism. They are fixed here because without them object identity, admissible stephood, and the distinction between continuation and repair are not stably assessable at all.

Definition 3.1 (Raw step-sequence)**.** A raw step-sequence on a domain D is an ordered sequence of inscriptions, rewrites, transitions, or operations on D. As such it carries no automatic claim to determinate target identity, admissible stephood, or preservation of object identity across the sequence.

Definition 3.2 (Kernel-neutral target adequacy)**.** A raw-sequence regime is target-adequate for the present inquiry iff some of its raw sequences are assessable under the following four requirements, stated without yet using the kernel as a definition of construction:

(A1) *Target determinacy:* the sequence has a repeatable object of evaluation rather than merely a presentation-dependent trace.

(A2) *Step evaluability:* its transitions are assessable as advancing, failing to advance, or departing from that same target rather than merely occurring after one another.

(A3) *Act-time finality:* a later intervention that changes the status of an earlier performance yields a distinct act or distinct sequence rather than retroactively making the same performance admissible.

(A4) *Same-regime fidelity:* identity-preserving redescription, encoding, or translation preserves target, step-status, and act-time classification; otherwise the presentation has changed regime or changed the object of evaluation.

These adequacy requirements are neutral with respect to the later vocabulary. They specify what must be preserved if a raw trace is to be assessed as construction of a determinate same-domain object at all. They are not proprietary terms of the present framework. They are the minimal commitments already incurred by any opponent who claims to have a counterexample to the paper's target: there must be something fixed, something done toward it, a stable distinction between the original act and later correction, and a preservation condition under redescription. A regime that denies any of these does not refute the target; it declines to instantiate it. Any regime that denies one or more of these conditions while still claiming to evaluate constructions of determinate same-domain objects has changed the target of inquiry rather than supplied a counterexample within it.

Lemma 3.3 (Target adequacy forces the kernel roles)**.** *Any target-adequate raw-sequence regime already requires the governance roles later named reference, standing, irreversibility, and admissibility.*

Proof. Target determinacy supplies the role later called reference: without a fixed target, there is no object whose construction could be assessed. Step evaluability supplies the role later called standing: without licit or illicit step-status, a transition is only a transition, not a construction step of the same target. Act-time finality supplies the role later called irreversibility: if later normalization can preserve the same act while changing its status, the difference between continuation and repair is not fixed at the time of performance. Finally, same-regime fidelity requires a boundary that holds those classifications together under identity-preserving redescription; that boundary is the role later called admissibility. Thus the kernel terms are introduced as names for forced roles, not as stipulative entry conditions. □

Remark 3.4 (Kernel terms as forced role names)*.* The kernel terms are introduced here only as names for roles whose necessity has already been fixed by the adequacy conditions. The downstream AASC machinery records consequences of this necessity rather than adding independent structure to it.

Lemma 3.5 (Kernel concession lemma)**.** *Any assessment that grants determinate target identity, evaluable stephood, act-time distinction between continuation and repair, and same-regime fidelity has already granted the kernel roles, whether or not it uses the vocabulary of AASC. Conversely, any assessment that denies one of those requirements no longer assesses a theorem-bearing construction of the same determinate endpoint. Therefore there is no third position from which one may reject the kernel while preserving ordinary theorem-bearing endpoint use for the same target.*

Proof. By Lemma 3.3, the four target-adequacy requirements force the roles later named reference, standing, irreversibility, and admissibility. Granting those requirements therefore grants the corresponding governance roles, independently of the terminology used to name them. If an assessment denies one of the four requirements, then by Definition 3.2 it no longer preserves the conditions under which a raw trace is assessable as construction of a determinate same-domain object. Such an assessment changes or exits the target class rather than preserving theorem-bearing endpoint use for the same target. The two cases are exhaustive: the requirements are either granted or denied. □

Definition 3.6 (Act identity)**.** Two performances count as the same act only if they agree on the admissibility-relevant invariant bundle, fix the same target on the same fixed domain, and have the same admissibility status at act-time. Any alteration that changes target, repairs inadmissibility, or reclassifies the admissibility status of the original performance yields a numerically distinct act or a distinct construction sequence.

Definition 3.7 (Governed construction and non-degenerate construction)**.** Let X be a target object on a fixed domain D. A governed construction of X, written $\mathrm{Constr}(X)$, is a raw step-sequence on D that preserves, at each step, determinate reference to X, standing of the step as an admissible construction step, and irreversibility of inadmissible steps. A governed construction is *non-degenerate* iff those three conditions hold together on the same fixed domain. A regime is *degenerate relative to the present target* iff it may still compute, transform, symbolize, or produce outputs but fails to preserve target identity, admissible stephood, act-time finality, or redescription fidelity as conditions of determinate same-domain construction.

This definition is classificatory rather than generative: it does not assume that all raw step-sequences already count as governed constructions, but distinguishes exactly those sequences for which admissibility-assessable construction is well-defined on the fixed domain.

Admissibility is not a fifth ingredient placed beside reference, standing, and irreversibility; it names the governance boundary under which those three conditions are jointly enforceable on the same sequence.

Meaningful regime. A regime is meaningful for the present target iff it is target-adequate in the sense of Definition 3.2 and some of its raw step-sequences are assessable as governed constructions of determinate objects rather than as bare symbol manipulations, encodings, or bookkeeping traces. This is not a stipulative filter. It is the least class in which object identity, admissible stephood, and act-time distinction between continuation and repair are available for assessment at all.

Fixed domain. A fixed domain is the domain on which target identity, identity-preserving transformation, and admissibility-relevant status are all evaluated. Operationally, two constructions remain on the same fixed domain iff there exists a translation between them that preserves (i) object identity under the admissibility-relevant invariant bundle, (ii) the admissibility classification invariant distinguishing admissible continuation from prohibited repair, and (iii) the class of governed constructions up to governance equivalence. A purported continuation remains on the same fixed domain only when these preservation conditions are met.

Why this target class is forced. The target class is not selected because it already contains the conclusion. It is the minimal class in which outputs are evaluable as determinate objects rather than presentation-dependent artifacts. By Lemma 3.3, the roles later named reference, standing, irreversibility, and admissibility are forced by target adequacy before governed construction is defined. If identity-preserving transformation is weakened, object identity becomes redescription-relative; if standing is weakened, there is no licit stephood rather than mere inscription; if irreversibility is weakened, the distinction between admissible continuation and retrospective rescue is unstable at act-time. A regime lacking these conditions may still support transformation, search, or computation, but it does not instantiate a weaker version of the same target phenomenon. The class is therefore fixed by evaluability conditions, not by convenience or stipulation.

Adequacy note. The question is not which extra norms we prefer to impose on an already functioning calculus. The question is what the weakest conditions are under which a raw step-sequence is assessable as construction of a determinate object at all. The next definitions and lemmas argue that stable reference, standing, and irreversibility are forced by that adequacy demand rather than borrowed from a thicker prior theory.

Scope restriction. All universal claims below are restricted to meaningful regimes in this adequacy sense. Systems lacking the adequacy conditions uncovered below are not counterexamples to the present thesis; they are different kinds of formal process because they lack the resources required for governed construction of a determinate same-domain object.

Definition 3.8 (Derivation)**.** When the target object X is a proposition or proof conclusion, the corresponding proposition-targeted special case of governed construction is called a derivation and is written $\mathrm{Der}(X)$.

Definition 3.9 (Governance equivalence)**.** Two governance conditions P and Q are governance-equivalent, written $P \simeq_{\mathrm{gov}} Q$, iff they perform the same governance work for identity-preserving construction on the same fixed domain; that is, replacing P by Q leaves the class of governed constructions unchanged.

Remark 3.10 (Equivalence as identity-level invariance). For the present target, governance-equivalence is not a loose behavioral similarity relation. Once admissibility-assessable, identity-preserving construction is fixed, admissibility classification is bivalent on acts and standing-bearing continuation is fixed by the same admissibility-relevant invariant bundle. Accordingly, any purported alternative governance structure that differs on any act, or on the admissibility-preserving continuation of any act, is not governance-equivalent on the fixed domain. Conversely, where no such divergence exists, the purported alternative contributes no distinct governance content for the target phenomenon.

The companion equivalence and conservation closures sharpen this point further: admissibility and standing do not form two merely coextensive tracks but collapse to the same standing-bearing classification, and conservation is not an auxiliary constraint but the invariant form of that classification. In that sense, equivalence collapses to identity at the level that matters here: any same-domain invariant that agrees with admissibility and standing on all acts and admissibility-preserving continuations is not merely extensionally similar but identical in governance content for admissibility-assessable construction.

Definition 3.11 (Admissibility-relevant invariant bundle)**.** An admissibility-relevant invariant bundle is any family of invariants sufficient to determine when two construction acts on a fixed domain preserve the same target, the same standing-bearing status, and the same admissibility boundary. Two acts or presentations count as belonging to the same fixed domain only when they agree on this invariant bundle up to governance equivalence.

Independent governance work. A condition performs independent governance work on a fixed domain iff its addition changes the set of governed constructions on that domain.

Same fixed domain. References below to "the same meaningful regime" are governed by preservation, not by mere reuse of vocabulary. Two presentations, encodings, or meta-systems count as the same fixed domain for the present target only when translation between them preserves determinate target, standing-bearing step status, the admissibility boundary between admissible continuation and prohibited repair, and the class of governed constructions up to governance equivalence. Failure of any one of these preservation conditions constitutes a change of domain rather than a redescription of the same one.

Definition 3.12 (Derivable from below)**.** A governance condition P is derivable from below iff there exists a governed construction of P whose admissibility can be certified using only governance conditions strictly independent of P, does not already presuppose P or any governance-equivalent condition, and does not rely on any condition whose own governance validity already requires P. In particular, approximation, convergence, limit, or emergent characterization of P counts as derivation from below only if the admissibility of the approximating, convergent, or emergent process itself does not presuppose P or any governance-equivalent condition.

The phrase "governed construction" in this definition is not meant to smuggle in the target conclusion. It marks the minimal condition under which the alleged lower derivation counts as a derivation rather than an unlicensed trace. A purported lower derivation may begin from any formal substrate it likes, but it must eventually explain how its own sequence obtains constructional license without using the role it claims to generate.

Definition 3.13 (Governance generation versus definability)**.** A condition is *defined* in a system when the system can write down, encode, or characterize it. A condition is *generated* in the present sense only when the system can produce that condition as new governance work from a lower basis. Definability without new governance work is not generation from below.

Definition 3.14 (Externally proposed construction or derivation system)**.** Let Σ be any purported construction system, derivation system, meta-theory, logical framework, encoding, or external presentation under which a statement P is said to be derivable. We say that Σ is *kernel-respecting with respect to* P iff the admissibility conditions under which sequences in Σ count as governed constructions already enforce P, some governance-equivalent condition, or some condition whose own admissibility already requires P. This definition classifies the admissibility conditions of Σ; whether a non-kernel-respecting system can still count as a system of non-degenerate construction for the present target is a theorem proved below, not a stipulation built into the definition.

Definition 3.15 (Kernel of non-degenerate construction)**.** The kernel of non-degenerate construction is the governance-role set

$$K := \{\mathrm{Adm}, \mathrm{St}, \mathrm{Ref}, \mathrm{Irr}\},$$

where Adm abbreviates admissibility, St standing, Ref reference, and Irr irreversibility. The minimality and uniqueness claims below concern governance roles and governance work on the fixed domain, not uniqueness of one four-part bookkeeping decomposition.

Remark 3.16. The point of the kernel definition is not to present four independent axioms, nor to shield four merely stipulated starting points from scrutiny. It is to name the lowest invariant cluster that any genuine governed construction already presupposes. Theorems below show that the members of K are neither optional nor independently eliminable.

Why standing is not redundant with admissibility. Admissibility marks gate status: whether an act may enter licensed construction. Standing marks act-carried eligibility for later licensed use, continuation, or commitment. The term "standing" is used because the relevant property is not merely that an act has passed a gate, but that it can stand as a source for subsequent licensed construction. The two coincide extensionally only after fixed-domain closure is proved in Theorem 3.66; they are not introduced as the same role. Before that closure theorem, admissibility answers whether the act is admitted, while standing answers whether the admitted act can function as a standing-bearing source for subsequent construction. The later collapse is therefore a result, not a definition.

Definition 3.17 (Same-regime faithful counterexample)**.** A same-regime faithful counterexample to the kernel is a target-adequate raw-sequence regime together with a class of candidate constructions such that all of the following are preserved on the same fixed domain: determinate target identity, step evaluability, act-time finality, identity-preserving redescription, and the class of standing-bearing continuations. The counterexample must nevertheless omit at least one member of K, and not merely replace it by a governance-equivalent role. Any example that loses one of the preservation conditions is a different formal process; any example that restores the missing work by an auxiliary selector, bridge, ranking, tolerance, or repair channel imports fresh governance rather than omitting the role.

Proposition 3.18 (Burden of a successful objection)**.** *To refute the kernel result for the present target, one must do at least one of two things: derive some $P \in K$ from below in the sense of Definition 3.12, or exhibit a same-regime faithful counterexample in the sense of Definition 3.17. Pointing to a different formal practice, a reversible search regime, a bookkeeping encoding, or a system with weaker target identity does not meet this burden.*

Proof. The target is fixed by target adequacy and same-domain fidelity. If an objection claims that a kernel role is generated rather than presupposed, then it must satisfy the strict lower-generation condition; otherwise the alleged derivation already uses the role it claims to produce. If the objection instead claims that the role is unnecessary, then it must preserve the same target phenomenon while omitting that role; otherwise it has changed the object of evaluation. Within the present target, a substantive challenge therefore alleges either lower generation or same-regime dispensability. The former is governed by derivability from below; the latter is governed by same-regime faithful counterexample. Anything else changes subject, imports structure, or supplies only notation. □

This burden is not an artificial raising of the evidential standard. It follows from target preservation. A counterexample to a necessity claim about determinate same-domain construction must preserve determinate same-domain construction. A regime that abandons target identity, step evaluability, act-time classification, or redescription fidelity is not a counterexample to the necessity claim; it is a different target.

Lemma 3.19 (Raw sequences are not yet constructions)**.** *A raw step-sequence does not count as a construction of a determinate object merely in virtue of being a sequence.*

Proof. A raw sequence may be written, executed, or transformed while leaving open what object, if any, it fixes; whether its steps are admissible steps of the same act; and whether later repair preserves or changes the act being assessed. Those are exactly the governance matters at issue. So raw sequentiality alone yields, at most, a trace. It does not yet yield construction of a determinate same-domain object. □

Lemma 3.20 (Same-act repair is impossible)**.** *If a later intervention changes an inadmissible performance into an admissible one, then the later intervention does not preserve the same act.*

Proof. By Definition 3.6, act identity requires preservation of the admissibility-relevant invariant bundle together with preservation of act-time admissibility status. A later intervention that converts inadmissibility into admissibility changes that status and therefore fails to preserve act identity. What results is either a distinct act or a distinct construction sequence, not repair of the same act. □

Lemma 3.21 (Reference is necessary for determinate construction output)**.** *If a purported construction act lacks determinate reference, then it does not yield a determinate object of the regime.*

Proof. A construction must be about or directed toward a target in a fixed way in order to preserve identity through its steps. If the act's expressions refer equivocally or vacuously, then no single governed target is fixed: different readings, or none at all, remain equally compatible with the same mark sequence. What remains is a trace, not a determinate object. Hence determinate construction output requires determinate reference. □

Lemma 3.22 (Standing is necessary for admissible stephood)**.** *If a purported step lacks standing, it does not function as an admissible construction step, but only as an unsupported inscription or transition.*

Proof. A construction step is not merely something that occurs between two inscriptions. It purports to advance a target under some licensed governance status. Without standing, there is no governed construction act whose role could be assessed. The item may still be written, computed, or recorded, but it does not count as an admissible step. Therefore standing is necessary for constructional stephood. □

Lemma 3.23 (Irreversibility is necessary for stable construction status)**.** *If invalid steps are repairable post hoc without loss, then the distinction between admissible continuation and retrospective rescue is not stably fixed at act-time.*

Proof. Suppose invalidity could later be removed while preserving the same act as fully admissible. By Lemma 3.20, that preservation claim is false: any intervention that changes inadmissibility into admissibility changes the act identity of what is being assessed. So if repair is allowed, the later admissible outcome is not the same act but a distinct act or distinct sequence. The original act therefore remains inadmissible, and any regime that treats later repair as preserving the same act loses a stable act-time distinction between continuation and rescue. Stable constructional status consequently requires irreversibility. □

Lemma 3.24 (Admissibility is the governance boundary of joint enforceability)**.** *Whenever determinate reference, standing, and irreversibility are jointly enforceable on the same construction acts, admissibility is already in force there as the governance boundary of that joint enforceability.*

Proof. By Lemmas 3.21–3.23, a regime that treats its acts as determinate, admissibility-assessable construction acts must already maintain determinate reference, licit step-status, and stable non-repairability of invalid steps. The point is not that one first has three free-standing ingredients and then adds admissibility as an extra fifth clause. Rather, once those three conditions are jointly enforceable on the same acts as conditions of construction, the regime already has a governance boundary distinguishing admissible from non-admissible acts and distinguishing continuation from retrospective rescue. That governance-boundary status is exactly what is here called admissibility. Conversely, a regime lacking that joint governance boundary does not govern construction as non-degenerate construction in the present sense. Hence admissibility is already in force wherever the three conditions are jointly enforceable. □

Proposition 3.25 (Boundary closure is forced by admissibility)**.** *Let a regime support admissibility-assessable, identity-preserving construction with irreversibility. Then any admissibility-relevant discriminator, selector, or classification rule must be invariant under identity-preserving transformation on the fixed domain.*

Consequently, any attempt to introduce additional admissibility-relevant structure not preserved under such transformations either:

(i) changes admissibility classification and so introduces a competing gate, or

(ii) leaves admissibility classification unchanged and is therefore inert bookkeeping.

Hence the admissibility classification invariant is closed under same-domain construction: no additional admissibility-relevant discriminator can be introduced without either collapsing to governance-equivalence or changing the domain of evaluation.

Proof. If an admissibility-relevant discriminator is not invariant under identity-preserving transformation, then two presentations of what purports to be the same target on the same fixed domain receive different admissibility significance. If that difference changes what counts as admissible continuation, standing-bearing reuse, or prohibited repair, then the regime has introduced a second

admissibility classifier for the same fixed-domain acts. But the distinction between admissible continuation and prohibited repair is exactly the governance work fixed by admissibility together with irreversibility; reclassifying same-domain acts therefore yields a competing gate rather than a mere refinement. If, by contrast, the additional discriminator does not change any admissibility verdict, then it contributes no independent governance work and is only bookkeeping. There is no third possibility: a same-domain addition either changes admissibility significance or it does not. Accordingly, admissibility-assessable, identity-preserving construction with irreversibility already forces closure of the admissibility classification invariant against further same-domain discriminators. □

Theorem 3.26 (Minimal assessability boundary)**.** *Let R be any system whose outputs are treated as determinate objects and whose transitions are evaluated as construction steps. Then determinate reference, admissible stephood, and non-repairable invalidity are jointly necessary and minimal for that evaluability. More exactly:*

(i) if determinate reference is removed, object identity is lost and the outputs are no longer evaluable as determinate content;

(ii) if standing is removed, no transition qualifies as an admissible construction step and the outputs are no longer evaluable as construction steps;

(iii) if irreversibility is removed, there is no stable distinction between admissible continuation and retrospective rescue, and the outputs are no longer evaluable as constructions; and

(iv) if an additional condition is imposed as an independent governance requirement beyond those three, then the regime is governed more tightly than evaluability itself requires.

Accordingly, these three conditions mark the minimal assessability boundary for identity-preserving construction as such. The minimality claim is a claim about governance work on the fixed domain, not about uniqueness of one bookkeeping decomposition.

Proof. Items (i)–(iii) are exactly the failure modes identified in Lemmas 3.21–3.23. Without determinate reference there is no fixed object identity and therefore no evaluable content; without standing there is no licensed act and therefore no evaluable construction step; without irreversibility there is no stable admissibility distinction and therefore no evaluable construction rather than retrospective normalization. The point is not merely that these failures are bad outcomes. It is that, once any one of them occurs, the target phenomenon itself is no longer present: what remains is trace-production, transformation, or bookkeeping rather than governed construction of a determinate same-domain object. This also blocks nearby weakenings and substitutes. Any purported weaker replacement for one of the three conditions must either preserve exactly the same governance work, in which case it is merely a notational variant of that condition; or else it must relax some part of that work, in which case one of the same excluded failures reappears—presentation-dependent object identity, indeterminate target, or collapse of the distinction between continuation and repair. If a purported weakening avoids those failures only by adding a new admissibility-relevant ranking, selector, tolerance, or repair-sensitive middle status, then it no longer weakens the package but supplements it by fresh governance structure. For minimality, suppose an additional condition Q is imposed as an independent governance requirement on every admissibility-assessable construction act. Then either Q is already enforced by the joint assessability work of reference, standing, and irreversibility, in which case it is not additional; or else Q excludes some act-sequences that already satisfy those three conditions, in which case Q governs more tightly than evaluability itself requires. There is no third possibility. So the three conditions are jointly necessary and minimal for that evaluability. □

Corollary 3.27 (Non-arbitrariness of the target class)**.** *The class of meaningful regimes is not defined by pre-selecting the kernel. It is exactly the class of systems in which determinate, admissibility-assessable construction is possible for the present target. Restriction to that class is therefore not protective but forced by the target of inquiry.*

Proof. By Theorem 3.26, determinate reference, standing, and irreversibility are not optional entry criteria appended in advance of the argument. They are the minimal conditions under which outputs and transitions are evaluable as construction at all. So to restrict attention to regimes in which those conditions are assessable is simply to restrict attention to the subject matter of the paper rather than to protect the thesis from counterexample by fiat. □

Theorem 3.28 (The target class is forced, not stipulated)**.** *Let R be any regime of raw step-sequences. If R lacks determinate reference, standing, or irreversibility in the senses fixed above, then R does not support governed construction of determinate same-domain objects. It supports only raw transformation, encoding, or bookkeeping processes.*

Proof. If determinate reference is lacking, Lemma 3.21 shows that no determinate target object is fixed. If standing is lacking, Lemma 3.22 shows that no step counts as an admissible construction step. If irreversibility is lacking, Lemma 3.23 together with Lemma 3.20 shows that act-time distinction between continuation and repair collapses. By Lemma 3.19, raw sequentiality does not by itself restore those losses. So a regime lacking any one of these conditions may still execute sequences, but it does not instantiate governed construction of determinate same-domain objects. □

Lemma 3.29 (Exhaustion of failure modes for non-kernel regimes)**.** *Let R be a regime of raw step-sequences. Suppose R lacks at least one member of K while claiming to support determinate, identity-preserving construction on a fixed domain. Then exactly one of the following holds:*

(i) object identity is not fixed, because determinate reference fails;

(ii) step status is not evaluable as admissible or inadmissible, because standing fails;

(iii) the distinction between continuation and repair is not stable at act-time, because irreversibility fails; or

(iv) the regime restores one or more of these distinctions only by adding additional governance-bearing structure not fixed on the original domain, and so proceeds by illicit import rather than same-domain construction.

These cases cover the preservation failures relevant to the present target.

Proof. If R lacks reference, then by Lemma 3.21 no determinate target is fixed, giving (i). If R lacks standing, then by Lemma 3.22 no step counts as an admissible construction step, giving (ii). If R lacks irreversibility, then by Lemmas 3.20 and 3.23 the distinction between admissible continuation and retrospective rescue is unstable at act-time, giving (iii). The only remaining possibility is that R attempts to preserve determinate identity, admissible stephood, or stable act-status while omitting one of the kernel roles. But then those distinctions are being reintroduced by fresh governance-bearing structure not fixed on the original domain. That is illicit import rather than a same-domain realization below the kernel, giving (iv). Within the present target, every purported non-kernel regime either loses one of the three assessability goods directly or restores it by extra same-domain authority. □

Lemma 3.30 (Reduction to kernel failure modes)**.** *Let R be any regime that deviates from the kernel while claiming to preserve determinate, identity-preserving construction on a fixed domain. Then every such deviation induces at least one of the following:*

(i) non-uniqueness of target under identity-preserving transformation;

(ii) indeterminacy of admissible stephood;

(iii) instability of admissibility under continuation; or

(iv) introduction of a selector, ranking, or auxiliary structure not invariant under identity-preserving transformation.

Hence every deviation reduces to a kernel failure mode.

Proof. If a deviation changes which target is fixed under identity-preserving transformation, then same-target identity is not preserved, giving (i). If it leaves target fixed but no longer preserves evaluable admissible stephood, then licensed step status becomes indeterminate, giving (ii). If it preserves target and stephood while allowing admissibility status to vary under purported same-act continuation, then the act-time distinction between continuation and repair is unstable, giving (iii). The only remaining possibility is that the regime preserves these goods by introducing some selector, ranking, tolerance class, or other auxiliary governance-bearing discriminator not already fixed by the original admissibility-relevant invariant bundle. But such a discriminator is fresh same-domain authority rather than preservation of the original regime, giving (iv). Since every deviation changes target, step-status, continuation-status, or the authority by which these are fixed, the partition covers the relevant ways a same-domain deviation can proceed. Therefore every deviation reduces to a kernel failure mode. □

Theorem 3.31 (Non-degenerate construction forces the kernel)**.** *If a meaningful regime admits non-degenerate construction in the sense of Definition 3.7, then the full kernel K is already in force in that regime. Equivalently, no meaningful regime supports non-degenerate construction in that sense while lacking admissibility, standing, reference, irreversibility, or some governance-equivalent replacement for one of them.*

Proof. Let R be a meaningful regime admitting non-degenerate construction in the sense of Definition 3.7. By Theorem 3.26, any regime in which outputs are assessable as determinate, admissibility-assessable constructions already requires determinate reference, admissible stephood, and irreversibility as the minimal assessability boundary of that evaluability. By Lemma 3.24, the joint enforceability of those conditions already places admissibility in force as the governance boundary constituted by their coherence on the same acts. Hence R already enforces Ref, St, Irr, and Adm, that is, the full kernel K. □

Corollary 3.32 (No non-degenerate regime below the kernel)**.** *No meaningful regime can both support non-degenerate construction in the sense of Definition 3.7 and generate any member of K from a strictly lower governance-free basis while remaining within that same fixed domain.*

Proof. Assume some meaningful regime R supports non-degenerate construction in the sense of Definition 3.7 and nevertheless generates some $P \in K$ from a strictly lower governance-free basis. By Theorem 3.31, the full kernel K is already in force in R before any such generation begins. So the alleged generator does not produce P from below; it operates only within a regime whose meaningfulness already depends on P. That contradicts the claim that P was first generated from a lower basis. □

Remark 3.33. The point of Theorem 3.31 is not to build the package into construction by stipulation. It is to show that, for the target domain fixed in the fixed-domain construction subsection, any regime that already supports non-degenerate construction already instantiates the kernel by necessity. The derivation lemmas of Section 3 therefore record a local reflection of a prior construction-level fact.

3.2 Presupposition Structure of Derivation

With Theorem 3.31 in place, derivation is the proposition-targeted special case of licensed construction. The present section records the derivation-level presupposition facts. These lemmas do not generate the package. They state what the fixed-domain construction subsection result looks like once the focus narrows from non-degenerate construction in general to derivation in particular: a sequence counts here as a derivation only if it already operates with admissibility, standing, reference, and irreversibility in place.

Lemma 3.34 (Derivation presupposes admissibility)**.** *If* $\mathrm{Der}(P)$ *holds for any proposition* P*, then admissibility is already in force.*

Proof. By Definition 3.8, a governed derivation is not merely a formal trace but a governed construction act whose target is a proposition. Its steps therefore count as steps only under an admissibility regime. So if $\mathrm{Der}(P)$ exists, admissibility has already been presupposed. □

Lemma 3.35 (Derivation presupposes standing)**.** *If* $\mathrm{Der}(P)$ *holds, then standing is already in force.*

Proof. A derivation with no standing would be a sequence of acts with no admissible target-bearing status. Such a sequence might be written down, but it would not count as a derivation of anything. Licensed derivation therefore presupposes standing. □

Lemma 3.36 (Derivation presupposes reference)**.** *If* $\mathrm{Der}(P)$ *holds, then determinate reference is already in force.*

Proof. A derivation whose expressions fail to refer determinately does not derive a determinate conclusion. It supplies marks, not governed content. Since $\mathrm{Der}(P)$ names a derivation of a proposition rather than a mere syntactic trail, reference is presupposed. □

Lemma 3.37 (Derivation presupposes irreversibility)**.** *If* $\mathrm{Der}(P)$ *holds, then irreversibility is already in force.*

Proof. Without irreversibility, invalid steps could be repaired retroactively into acceptable ones, and the distinction between admissible continuation and post hoc rescue would collapse. In that case the derivation would not possess a stable governance status at all. Hence governed derivation presupposes irreversibility. □

Corollary 3.38 (Global presupposition lemma)**.** *For every proposition* P*, if* $\mathrm{Der}(P)$ *holds, then the full kernel* K *is already in force.*

Proof. Combine the four previous lemmas. □

Theorem 3.39 (Definability is not governance generation)**.** *Let* P *be any governance condition. A system may define, encode, or represent* P *without generating* P *from below. Generation from below occurs only if the system contributes new governance work sufficient to produce* P *for the original acts on the original fixed domain.*

Proof. A definition or encoding may restate a governance condition, give it a symbol, embed it in a meta-language, or characterize its extension. None of those activities by itself changes the class of governed constructions on the original fixed domain. By definition, governance generation requires independent governance work. So definability, representation, and coding are not by themselves generation from below. □

Remark 3.40. Accordingly, the non-derivability claims below are not claims that one cannot write down or characterize admissibility, standing, reference, or irreversibility. They are claims that no lower same-domain basis generates the governance work those conditions perform for the original acts.

3.3 Non-Derivability of the Kernel

The results of this section have two directions. Internally, they show that no lower same-domain basis generates the governance work performed by the kernel. Externally, they identify the construction-level form of a broader meta-necessity: in any non-degenerate irreversible compositional regime, the invariant required to make irreversibility well-defined is exactly the invariant characterized here under the fixed-domain conditions of identity-preserving construction. The kernel is therefore not an optional refinement layered on top of admissibility. It is the minimal decomposition of the admissibility invariant once the target object is fixed.

The argumentative standard in this section is deliberately adversarial. A proposed defeat of the kernel must either supply a faithful lower generator for one of its roles, or supply a same-regime faithful counterexample that preserves the target phenomenon while lacking that role. The former fails because any licensed construction of the role already relies on the role or a governance-equivalent substitute. The latter fails because any same-regime omission either loses target identity, loses licit stephood, loses act-time irreversibility, imports a replacement selector, or collapses extensionally back to the same governance work.

Theorem 3.41 (Non-derivability of admissibility)**.** *Admissibility is not derivable from below.*

Proof. Assume for contradiction that admissibility is derivable from below. Then there exists a governed construction of Adm whose admissibility does not already presuppose Adm or a governance-equivalent condition. But by the previous section, any licensed construction already presupposes admissibility. So the alleged construction uses what it claims to produce. This is precisely the forbidden circularity in the definition of derivability from below. Therefore admissibility is not derivable from below. □

Theorem 3.42 (Non-derivability of standing)**.** *Standing is not derivable from below.*

Proof. Suppose standing were derivable from below. Then there would exist a governed construction of St whose admissibility did not already presuppose standing. By the previous section, however, the very existence of such a construction already presupposes standing. The construction therefore fails the "from below" condition by collapsing into the very status it was supposed to generate. □

Theorem 3.43 (Non-derivability of reference)**.** *Reference is not derivable from below.*

Proof. If reference were derivable from below, there would be a governed construction of Ref whose admissibility did not already rely on determinate reference. But by the previous section, no licensed construction exists without determinate reference. Hence the putative construction again presupposes what it claims to derive. Reference is therefore not derivable from below. □

Theorem 3.44 (Non-derivability of irreversibility)**.** *Irreversibility is not derivable from below.*

Proof. Assume irreversibility is derivable from below. Then a governed construction of Irr exists whose admissibility does not already presuppose irreversibility. The previous section rules this out: a licensed construction already relies on the stable difference between admissible continuation and invalid repair. So irreversibility cannot be derived from beneath itself. □

Corollary 3.45 (Kernel non-derivability)**.** *No member of K is derivable from below.*

Proof. Immediate from the four preceding theorems. □

Theorem 3.46 (No faithful lower generator / structural non-derivability)**.** *Let $P \in K$. No system supporting non-degenerate construction can generate P from a genuinely lower condition while remaining within the same fixed domain. Equivalently, any such system already has P in force before internal construction begins.*

Proof. Assume for contradiction that some system or meaningful regime R supports non-degenerate construction in the sense of Definition 3.7 and nevertheless generates some $P \in K$ from a purportedly lower package. There are only two cases.

First, the alleged generator operates on the same fixed domain as the original acts. Then by Theorem 3.31, the full package K is already in force in R, and hence P is already in force there. So the alleged lower package does not generate P from below; it operates only inside a regime whose meaningfulness already depends on P. In this same-domain case the remaining escape routes are exhausted. A purported lower generator must either (i) introduce additional same-domain structure not already licensed on the fixed domain, (ii) collapse to triviality by adding no independent governance work, (iii) alter identity-preserving transformation, reference conditions, or act-identity conditions so that the original target phenomenon is no longer preserved, or (iv) reproduce the original admissibility boundary extensionally under a different presentation. Route (i) is illicit same-domain import rather than lower generation. Route (ii) is not generation at all. Route (iii) changes domain rather than generating the kernel within the same one. Route (iv) is extensional redescription of what was already in force. Any further same-domain route would have to do independent governance work while avoiding all four branches, which is precisely what the preceding results exclude.

Second, the alleged generator operates outside the original fixed domain and then attempts to re-enter it. But then the resulting construction does not govern the original acts on their original fixed domain without an additional transfer principle, selector, or authority connecting the external regime back to the old one. That additional bridge is exactly fresh structure not fixed by the original domain. More sharply: the admissibility boundary together with standing classification fixes the identity of the domain under evaluation. So any purported derivation that changes that boundary or that classification has not generated the kernel for the original domain; it has changed the object of evaluation and is therefore a scope change rather than a lower derivation. The maneuver is consequently scope change or illicit import rather than lower generation of the kernel for the original acts.

Thus neither same-domain nor cross-domain generation succeeds. Re-describing the alleged generator as belonging to a regime that lacks P therefore does not help, because either it fails to govern the original domain or, if it does govern it, P was already in force there. The obstruction is structural rather than a peculiarity of one proof format. □

Theorem 3.47 (Regime-invariant non-derivability)**.** *Let $P \in K$. No system Σ can derive P from below unless the admissibility conditions under which a Σ-construction counts as a construction already enforce P, something governance-equivalent to P, or a condition that already requires P.*

Proof. Let $P \in K$, and suppose some system Σ is offered as a construction or derivation system for P. There are two cases.

First, the admissibility conditions under which a Σ-construction counts as a licensed construction already enforce P, a governance-equivalent condition, or a condition whose own admissibility already

requires P. Then Σ is kernel-respecting with respect to P by Definition 3.14, and the alleged construction does not generate P from below.

Second, those conditions do not already enforce P in any of those ways. If Σ nevertheless supported non-degenerate construction in the same fixed domain, Theorem 3.31 would force the whole package, including P, already to be in force there. So a non-kernel-respecting Σ cannot both lack P and still count as a system of non-degenerate construction for the present target. Its output is therefore, at best, an external coding, a bookkeeping device, or a purely formal trace; it is not a construction of P from below for the target phenomenon at issue.

Changing formal dress, moving to a meta-theory, or recoding the language therefore does not remove the need for the work that P names. By Theorem 3.39, such recodings may define or represent P without generating it from below. □

Theorem 3.48 (No faithful lower generator and no faithful same-domain extension)**.** *For the admissibility boundary on a fixed domain, there is no faithful lower generator and no faithful same-domain extension that converts admissibility, standing, reference, or irreversibility into lower theorems for the original acts on the original fixed domain. Any purported attempt ends in exactly one of four outcomes: illicit structure import, triviality, violation of the base goods, or extensional collapse back to the gate itself.*

Proof. Any genuinely pre-boundary constraining device must be available and meaningful on the fixed pre-admissible side. If it is not, it is illicit import. If it is, then under the same-domain invariance conditions relevant here it either has no independent constraining force on individual acts, destroys reference, identity, or irreversibility, or coincides extensionally with the gate already being determined. These four branches cover the ways a purported pre-boundary device could relate to the original acts. A further case would have to contribute independent same-domain governance work while neither importing structure, trivializing the construction, violating the base goods, nor collapsing extensionally to the gate. But independent same-domain governance work is exactly what the non-derivability and no-faithful-same-domain-extension results exclude. The same exhaustion therefore remains in force under faithful same-domain extension, because fresh same-domain authority for the original acts is itself illicit structure import unless it changes nothing or collapses back to the original gate. □

Proposition 3.49 (Cross-domain closure and admissibility transport)**.** *The non-derivability results above exclude not only same-domain lower generators but also cross-domain simulation attempts. In particular, let Σ be any external system and let ϕ be a mapping from Σ into the fixed domain of the original acts. If ϕ preserves (i) target identity under identity-preserving transformation, (ii) admissibility classification, and (iii) standing-bearing continuation, then ϕ is governance-equivalent to the original domain on the relevant image. If ϕ fails to preserve any of (i)–(iii), then the attempted transport changes the admissibility-relevant invariant bundle and constitutes a scope change rather than a derivation. Accordingly, cross-domain simulation does not provide a lower generator for admissibility on the fixed domain: it either collapses to governance-equivalence or exits the domain of evaluation.*

Proof. The fixed domain is identified by preservation of target identity, admissibility classification, and the class of governed constructions up to governance equivalence (Definition 3.7 and the fixed-domain clause of the fixed-domain construction subsection). By Proposition 3.25, admissibility-assessable, identity-preserving construction with irreversibility already forces closure of the admissibility classification invariant against further same-domain discriminators. So if a mapping ϕ from an external system preserves target identity, admissibility classification, and standing-bearing

continuation for the original acts, then it preserves exactly the invariant bundle that individuates the fixed domain for the present target. On the relevant image, ϕ therefore contributes no independent governance work beyond what is already counted as the same domain; it is governance-equivalent rather than a lower generator.

If, by contrast, ϕ fails to preserve any one of those items, then the transported construction no longer governs the original acts as the same fixed-domain objects with the same admissibility classification and standing-bearing continuation. That is not derivation of admissibility for the original acts, but change of the object under evaluation. Hence cross-domain transport either collapses to governance-equivalence or exits the domain of evaluation. □

Remark 3.50. The previous results do not show that one may never write down statements asserting admissibility, standing, reference, or irreversibility. They show something sharper: these conditions cannot be generated as foundations by a construction that does not already rely on them. Their non-derivability is thus a minimality result, not an expressive prohibition.

Theorem 3.51 (No governance-effective deeper same-domain invariant)**.** *Let a system support admissibility-assessable, identity-preserving construction with irreversibility. Then no governance-effective invariant strictly beneath admissibility exists for that same fixed-domain target.*

More exactly, any purported invariant that claims to ground, constrain, or underwrite admissibility must fall into exactly one of the following cases:

(i) it imports structure not invariant under identity-preserving transformation and therefore changes domain rather than grounding admissibility on the fixed domain;

(ii) it carries no constraining power on admissibility-assessable construction and is therefore vacuous with respect to the present target; or

(iii) it is governance-equivalent to admissibility or standing under renaming and therefore performs no deeper governance work.

Accordingly, no governance-effective deeper same-domain invariant remains compatible with irreversible, identity-preserving construction on the fixed domain.

Proof. Any purported deeper invariant must, by hypothesis, do one of two things: either it constrains admissibility, or it fails to constrain it. If it fails to constrain admissibility, then it does no governance work for the target phenomenon and is vacuous relative to identity-preserving construction. So only the constraining case remains.

Now any invariant that constrains admissibility must be available wherever admissibility is being fixed for the same acts on the same domain. If it is introduced only by adding structure not preserved under identity-preserving transformation, then it does not underwrite admissibility on the fixed domain but changes the admissibility-relevant invariant bundle and thus changes domain. If instead it is presented as same-domain and same-target while avoiding such imported structure, then by the non-derivability and no-faithful-same-domain-extension results above, it cannot contribute independent deeper governance work beneath or prior to admissibility. In that case either it leaves the admissibility classification unchanged, in which event it is inert bookkeeping, or it reproduces the same governance work already performed by admissibility or standing under a different presentation, in which event it is governance-equivalent under renaming.

A further option would have to constrain admissibility on the same fixed domain while neither importing structure, nor being vacuous, nor collapsing to governance-equivalent admissibility work. But that is exactly the kind of independent deeper same-domain invariant excluded by the preceding non-derivability results together with Proposition 3.49 and the fixed-domain-extension result. Therefore no governance-effective invariant strictly beneath admissibility exists for the present fixed-domain target. □

Remark 3.52 (Dependency status of invariant impossibility). Here and throughout, "deeper invariant" means any invariant that purports to ground or constrain admissibility without being reducible to it.

The preceding theorem is eliminative, not generative. It does not serve as a premise for the existence or necessity of admissibility. Rather, it constrains the space of possible alternatives once admissibility is already required by irreversibility and identity-preserving construction and once the fixed-domain kernel has been established in the kernel forcing and non-derivability subsections.

In particular, the impossibility of deeper invariants is not used to derive admissibility itself, but only to exclude competing invariant structures that might otherwise be proposed in its place. Any dependence on architectural commitments such as non-transmissive closure or boundary discipline is therefore downstream: within the argumentative order of the present paper, those commitments are consequences of the admissibility structure already established in the kernel forcing and non-derivability subsections or eliminative restatements of what those sections exclude, not independent assumptions on which the kernel results rely.

3.4 Mutual Closure and Minimality

Proposition 3.53 (Admissibility requires standing)**.** *Any regime lacking standing is not an admissibility regime.*

Proof. A regime without standing does not distinguish licit construction from unsupported utterance or transition. It therefore lacks the governance force required to count as admissibility. At best it describes transitions; it does not govern construction. □

Proposition 3.54 (Standing requires reference)**.** *Standing without determinate reference is incoherent.*

Proof. Standing is the licitness of a construction act. But where reference is indeterminate, there is no determinate target whose licitness could be assessed. A purported standing relation with no reference relation underneath it licenses emptiness or equivocation, not governed construction. □

Proposition 3.55 (Governed reference requires irreversibility)**.** *The reference relevant to non-degenerate construction is not bare denotation but reference fixed for governed use at act-time. That form of reference cannot be maintained if invalid construction acts are repairable post hoc without loss.*

Proof. A symbol may still bear a denotational relation under later reinterpretation or repair. That is not yet the kernel notion of reference. The issue here is whether the act, as performed, fixes a determinate target for governed constructive use. If later repair can retroactively convert the same invalid act into a fully admissible one without loss, then the constructive status under which the act's target is fixed is not settled at act-time. What remains may be denotation, but not governed reference in the sense required for non-degenerate construction. Hence kernel-reference requires irreversibility of invalidity. □

Proposition 3.56 (Governance irreversibility requires admissibility)**.** *The irreversibility relevant here is not mere temporal one-wayness or procedural non-rewindability. It is the governance distinction between licensed continuation and prohibited repair, and that distinction is available only under an admissibility boundary.*

Proof. One may define other senses of irreversibility—causal, temporal, computational, or procedural—without invoking admissibility. Those are not the senses at issue here. The kernel member Irr concerns irreversible invalidity of construction acts. Without admissibility there is no principled governance boundary under which one continuation counts as licensed while another counts as illicit repair. One can still speak of sequence or alteration, but not of irreversible invalidity in the governance sense. Thus governance irreversibility presupposes admissibility. □

Theorem 3.57 (Mutual closure of the kernel)**.** *The members of K form a mutually closed necessity set: none can be coherently asserted while denying the others' governance role.*

Proof. Propositions 3.53–3.56 exhibit a governance-specific necessity loop: admissibility requires standing, standing requires reference, governed reference requires irreversibility, and governance irreversibility requires admissibility. Hence no member can be detached from the rest while retaining its intended governance content. What remains after detachment is either vacuous, equivocal, or degenerate. □

Remark 3.58. The last two arrows of the loop concern the governance senses fixed in the fixed-domain construction subsection. Proposition 3.55 is not a claim about bare denotation independently of construction; it concerns reference as fixed for governed constructive use at act-time. Proposition 3.56 is not a claim about every temporal or procedural notion of irreversibility; it concerns irreversible invalidity of construction acts. In those senses, the loop is closed.

Theorem 3.59 (Minimality of the kernel)**.** *The kernel K is minimal for non-degenerate construction: no proper subset of K suffices for non-degenerate construction.*

Proof. Assume a proper subset $K_0 \subsetneq K$ suffices for non-degenerate construction. Let $X \in K \setminus K_0$ be an omitted member. Since non-degenerate construction allegedly remains possible, either X is unnecessary or X is generated from the remaining members. The first option contradicts Theorem 3.57, since the governance role of each member depends on the rest. The second option contradicts Corollary 3.45, since X would then be derivable from below from a lower basis. Both possibilities fail. Therefore no proper subset suffices. □

Corollary 3.60 (No lower generator)**.** *There exists no governance condition or package strictly below K that generates K.*

Proof. If such a lower generator existed, at least one member of K would be produced from below by a package not already equivalent to that member. That contradicts Corollary 3.45. □

Theorem 3.61 (Uniqueness of governance work, not of bookkeeping decomposition)**.** *If K' is any governance package sufficient for non-degenerate construction in the sense of Definition 3.7, then K' must cover the same governance roles as K and is governance-equivalent to K at that role level. Coarser or finer decompositions may exist; what cannot exist is a governance-non-equivalent alternative package, where governance-non-equivalent means inducing a different class of governed constructions.*

Proof. Any package sufficient for non-degenerate construction must, by Theorem 3.31, already secure admissibility, standing, reference, and irreversibility, or governance-equivalent replacements for those roles. If it fails to supply any one of those governance roles, non-degenerate construction collapses by Theorem 3.57. A coarser package may compress multiple roles into one clause, and a finer package may split one role across several clauses. But if the total governance work and the resulting class of governed constructions remain unchanged, then by Definition 3.9 the package

is governance-equivalent to K at the level that matters here. If the governance work changes, the package is governance-non-equivalent and is not sufficient for non-degenerate construction in the present sense. Therefore the uniqueness claim concerns required governance work, not the uniqueness of one four-part bookkeeping decomposition. □

Remark 3.62. This theorem concerns governance-role equivalence, not representational identity and not uniqueness of a particular factorization. It does not exclude alternative formal encodings, type-theoretic or categorical implementations, game-semantic presentations, or coarser/finer decompositions that preserve the same governance work. It excludes only packages that alter the class of governed constructions and therefore perform different governance work.

3.5 Fixed-Domain Closure Results

In the present section, "same domain" always means preservation of the admissibility-relevant invariant bundle and hence preservation of the class of governed constructions up to governance equivalence. The point is not to classify arbitrary statuses or metrics, but to describe what fixed-domain admissibility can and cannot look like once the kernel is in place.

The bivalence and AMetricity claims below concern only gate status for standing-bearing commitment, licensed reuse, and admissible continuation. They do not deny object-language many-valued semantics, paraconsistent valuations, probabilistic confidence, proof-search rankings, approximation orders, proof-state metadata, type refinements, or graded measures internal to an already admitted construction regime. Such structures are admissible only when they do not themselves decide standing-bearing entry. If they do decide entry, they are part of the admissibility gate and fall under the present results; if they do not, they are downstream bookkeeping or internal structure.

These results do more than describe one convenient interface. Taken together, they close the remaining underdetermination question for the admissibility invariant itself. Once a fixed domain already requires an admissibility boundary, every same-domain realization must either reproduce the interface proved in this section up to governance-equivalence or else fall into one of the failure modes already excluded: hidden selector work, repair-sensitive middle status, illicit structure import, domain change, or inert bookkeeping. The point of the section is therefore structural rigidity: necessity plus uniqueness of same-domain realization.

Lemma 3.63 (No intermediate admissibility state)**.** *Let S be any admissibility-relevant status distinct from admitted and not-admitted on a fixed domain. Then exactly one of the following holds:*

(i) S affects admissible continuation, commitment, or licensed reuse, in which case S performs hidden selector work or reopens a repair-sensitive middle status; or

(ii) S affects none of those, in which case S is inert bookkeeping rather than an admissibility-relevant status.

Hence no third admissibility-relevant status exists.

Proof. Take any status S distinct from admitted and not-admitted. If possession of S changes what may later count as licensed continuation, commitment, or reuse, then S adds governance-relevant discrimination beyond the gate itself. In that case it either selects among otherwise eligible acts or preserves a repair-sensitive middle state between rejection and admission. If instead possession of S changes nothing about later licensed use, then S contributes no governance work and is merely descriptive bookkeeping. Since every admissibility-relevant status must either affect licensed use or fail to affect it, these two cases are exhaustive. Therefore no third admissibility-relevant status survives. □

Theorem 3.64 (Necessity and bivalence of admissibility)**.** *On every fixed domain supporting licensed construction, a prior admissibility predicate is forced, and any admissibility-relevant status interface on that same fixed domain is extensionally equivalent to a binary one. Equivalently, every admissibility-relevant status falls into exactly one of two classes: admitted for commitment, reuse, and standing-bearing continuation, or not admitted.*

Proof. Without a prior admissibility predicate, entry into licensed composition would have to occur before evaluation, by post hoc repair, by scope transport, or by injected parameterization. Each route breaks reference, composition, or irreversibility. So a prior gate is forced. Now suppose the resulting admissibility interface were not extensionally binary. Then some additional same-domain status would have to survive between, or alongside, admitted and not admitted. If that status changes later admissible use, it is hidden gate work or a repair-sensitive middle state and therefore contradicts the fail-closed consequences of irreversibility. Any state whose admissibility status can change without constituting a numerically distinct act violates irreversibility and is therefore excluded. If it changes no later admissible use, it is inert bookkeeping and not an admissibility-relevant status at all. There is no third possibility. So every admissibility-relevant interface collapses extensionally to admitted or not admitted. □

Theorem 3.65 (AMetric boundary and non-parameterization)**.** *At the admissibility boundary, no admissibility-relevant selector, ranking, ordering, grading, metric, or discrete indexing survives. In particular, there is no admissibility-relevant family of distinct positive statuses beyond the binary admitted / not-admitted split.*

Proof. On the pre-boundary side, no representative, coordinate, or labeling choice may carry admissibility-relevant authority for the same fixed domain. This label-neutrality is not a free modeling preference. If admissibility were allowed to depend on labels, coordinates, or presentation indices as such, then two identity-preserving presentations of the same candidate would receive different admissibility significance solely by representation. That would make representation itself a same-domain selector and would violate the requirement that admissibility be invariant under identity-preserving transformation. So any admissibility-relevant differentiator must be invariant under governance-preserving relabeling. Under that invariance condition, only equality-pattern structure survives: there is no nontrivial unary grading, no admissibility-relevant order, no selector of a privileged positive route, and no meaningful metric beyond discrete equality. Suppose, then, that a parameterized or ranked boundary nevertheless existed. If the parameter or ranking makes a governance-relevant difference, it selects among otherwise admissible candidates or routes and therefore introduces a same-domain selector not preserved by the relabeling symmetry just described. If it makes no governance-relevant difference, it is bookkeeping rather than boundary structure. Likewise, if the extra structure is used to encode an intermediate positive status rather than a selector, it reopens a repair-sensitive middle state between admitted and not admitted, contrary to the fail-closed consequence of irreversibility. Thus every purported admissibility-relevant parameterization or metric either contradicts the boundary invariance conditions or collapses to bookkeeping. Accordingly, any purported admissibility-relevant parameterization at the boundary is forbidden. □

Theorem 3.66 (Standing equals reuse-stable admissibility)**.** *Let* Stand_D *be any predicate claimed to mark those evaluated acts on a fixed domain* D *that are eligible for standing-bearing continuation, commitment, or licensed reuse. Then for every act* $a \in D$*,*

$$\mathrm{Stand}_D(a) = 1 \iff a \text{ is reuse-stably admissible on } D.$$

Equivalently, standing is not an extra primitive or interface-level classifier beyond admissibility on a fixed scope. It is the persistence condition of admissibility under licensed same-scope continuation.

Proof. If $\mathrm{Stand}_D(a) = 1$, then standing is already doing admissibility-relevant gate work on the fixed scope, so by Theorem 3.64 there is no additional governance-relevant classifier beyond the unique gate. Thus a is admitted, and if it were not reuse-stable some licensed same-scope continuation would silently retarget or reclassify the act. Conversely, if a is reuse-stably admissible but denied standing, then admitted acts are split into two substantive positive subclasses. If that split affects later continuation, it is forbidden same-side gate work; if it affects nothing, it is inert bookkeeping. So the split is impossible as a substantive classification. □

Theorem 3.67 (Extensional uniqueness of the admissible interior)**.** *For a fixed system S on a fixed domain, the admissible interior is unique up to extensional, or lawful, equivalence. More precisely, if I_1 and I_2 are predicates on acts such that each contains exactly the standing acts of S, then for every act a,*

$$I_1(a) \iff I_2(a).$$

Equivalently: in the standing-instantiated regime, there are no plural admissible interiors in extension; any adequate encoding of the admissible interior collapses pointwise to the standing predicate.

Proof. Assume that I_1 and I_2 both capture exactly the standing acts on the same fixed domain but that they disagree on some act a. Then the same act receives two different standing verdicts on the same fixed scope. If one now adds a selector deciding which interior governs a, the selector itself becomes a new same-domain admissibility-relevant parameter, contrary to Theorems 3.64 and 3.65. If one adds no such selector, then the fixed-domain standing classification is indeterminate, contrary to the role of standing in Theorem 3.66. So two extensional interiors capturing exactly the standing acts cannot disagree on any act. Their apparent plurality therefore collapses extensionally to one admissible interior. □

Theorem 3.68 (Conservation of standing)**.** *For each fixed domain, no internal operation of the licensed construction regime creates, amplifies, transfers, or recovers standing. Standing can only be preserved or destroyed.*

Proof. Recovery would be same-act repair; creation would be standing from a failed act without fresh evaluation; amplification would require standing to admit degrees despite Theorem 3.64; and transfer would require an independent carrier of standing distinct from the gate. Each branch is excluded by the preceding kernel packet. So only preservation or destruction remain. □

Theorem 3.69 (Structural rigidity of the admissibility invariant)**.** *Let D be any fixed domain supporting governed construction. Let J_D be any candidate same-domain realization of the admissibility invariant sufficient to fix eligibility, standing-bearing continuation, and reference-preserving commitment on D. Then exactly one of the following holds:*

(i) J_D is governance-equivalent to the interface already fixed by the present results: a binary admissibility gate with an AMetric boundary, standing equal to reuse-stable admissibility, a unique admissible interior, and conservation of standing;

(ii) J_D introduces fresh same-domain selector work, repair-sensitive middle status, or other illicit structure and therefore fails as a faithful realization on D;

(iii) J_D changes the admissibility-relevant invariant bundle and therefore changes domain rather than realizing the same invariant on the same domain; or

(iv) J_D contributes no independent governance work and collapses to bookkeeping.

In particular, the admissibility invariant required for standing-bearing construction admits no governance-non-equivalent same-domain realization.

Proof. By Theorem 3.61, any package sufficient for non-degenerate construction must already cover the same governance roles as K up to governance-equivalence; there is no further governance-non-equivalent package that preserves the same class of governed constructions. Theorems 3.64 and 3.65 fix the interface shape of any such package on a fixed domain: admissibility-relevant status collapses extensionally to admitted or not admitted, and no same-domain selector, ranking, metric, or parameter survives at the boundary. Theorem 3.66 then eliminates any additional positive-side classifier beyond reuse-stable admissibility; Theorem 3.67 eliminates plural same-domain admissible interiors; and Theorem 3.68 eliminates creation, amplification, transfer, and same-act recovery of standing.

Suppose, then, that J_D is a candidate realization of the required invariant on the same fixed domain. If it preserves all of the governance work just listed, then by Theorem 3.61 it is governance-equivalent to the interface already proved, giving (i). If it differs by introducing fresh selector work, middle statuses, repair channels, or imported authority, then it contradicts Theorems 3.64, 3.65, and 3.68, yielding (ii). If it preserves some other boundary only by altering what counts as the same target, the same standing-bearing continuation, or the same admissibility-relevant invariant bundle, then it is no longer realizing the same invariant on the same fixed domain, yielding (iii). If it changes no governance verdict at all, then it is an extensional redescription or bookkeeping layer, yielding (iv). Any further candidate would have to change governance work while avoiding the listed branches, which is exactly what the rigidity result excludes. □

3.6 Consequences for Admissible Structure

The previous sections establish that the foundational layer is fixed, minimal, and non-derivable. The present section derives several structural consequences once that package is in place. These are not additional assumptions; they are strict consequences of the fixed package itself. All consequences in this section follow strictly from the impossibility of same-domain selector introduction, repair-sensitive status, or invariant violation established in Sections 4–6. The forcing pattern is uniform. A purported same-domain alternative to the results below must either (i) introduce an additional admissibility-relevant parameter or selector, (ii) reopen a repair channel or middle status between admission and rejection, or (iii) preserve all same-domain governance work and therefore collapse extensionally to the original interface. The first two branches contradict the kernel results already established; the third yields only bookkeeping redescription. The arguments below therefore proceed by contradiction against those three possibilities rather than by stipulating an interface in advance.

Definition 3.70 (Invariant bundle and same target)**.** An invariant bundle is a family of admissibility-relevant invariants sufficient to determine when two acts have the same governed target. For acts a and b, write $a \sim b$ to mean that they agree on every invariant in the bundle.

Definition 3.71 (Scope-preserving continuation)**.** A continuation C is scope-preserving iff it remains within one admissibility-relevant frame and does not change target by illicit redescription.

Definition 3.72 (Admissible operator)**.** An operator or relation is admissible iff its application respects the governance of standing and admissibility rather than introducing distinctions by redescription alone.

Theorem 3.73 (Scope-preserving invariance)**.** *If C is scope-preserving, then for every admissible act a, one has $a \sim C(a)$.*

Proof. Assume C is scope-preserving but changes an admissibility-relevant invariant of a. Then a and $C(a)$ do not fix the same target. The continuation would therefore have introduced a governance-bearing difference while still claiming to remain within the same scope. That is precisely a silent redescription or hidden reparameterization, which admissibility excludes. Hence every scope-preserving continuation preserves the invariant bundle pointwise. □

Theorem 3.74 (Transport closure)**.** *Let R be an admissible relation. If $b_1 \sim b_2$, then for every q,*

$$R(q, b_1) \iff R(q, b_2).$$

Proof. If R could distinguish b_1 from b_2 despite $b_1 \sim b_2$, then the distinction would arise from presentation rather than governed target. But admissible construction cannot allow standing-bearing content to depend on mere redescription. Once target is fixed, admissible transport must be closed under that fixing. Therefore admissible relations cannot distinguish same-target presentations. □

Theorem 3.75 (Domain-defined operators)**.** *An admissible operator is defined exactly on the admissible domain, and in particular must be defined on every standing point applied to itself.*

Proof. If an admissible operator were undefined on a standing point applied to itself, then a standing act would fail to lie in its own governed domain. The operator would not govern the admissible act as such. Conversely, if the operator had governance-bearing force outside the admissible domain, it would confer structure where standing is absent and thereby violate the admissibility boundary. Hence admissible operators are defined exactly on the admissible domain. □

Theorem 3.76 (Quotient identity)**.** *Identity of admissible presentation is exactly same-target identity. Equivalently, the admissible redescription orbit of an act coincides with its invariant bundle class.*

Proof. Suppose two presentations lie in the same admissible orbit but differ in same-target identity. Then admissible redescription would change governed target, violating Theorem 3.73. Conversely, suppose two presentations agree on every admissibility-relevant invariant but fail to count as identical for admissible purposes. Then admissibility would preserve a surplus distinction not grounded in target, violating Theorem 3.74. Therefore orbit identity and invariant identity coincide. □

Corollary 3.77 (No silent redescription)**.** *Any purported continuation that changes admissibility-relevant status while claiming same scope is illicit.*

Proof. A same-scope continuation must preserve the invariant bundle by Theorem 3.73. A status-changing continuation would therefore either leave scope or introduce illicit redescription. In either case it is not admissible as same-scope transport. □

Remark 3.78. The formal consequences listed here—transport closure, quotient identity, domain-defined operators, and scope-preserving invariance—therefore do not stand as fresh assumptions. They are structural fallout from the fixedness of the kernel. They belong downstream of foundational closure, not alongside it.

3.7 Closure Against Additional Foundational Conditions

In this section, an *additional foundational condition* means a putative further bearer of irreducible constructional work. It does not mean a merely declared symbol, shorthand, or presentational starting point. The question is whether anything beyond K can function as a further base-level condition of non-degenerate construction.

Any such purported further condition can enter only in one of three ways: it can refine admissibility classification on the same fixed domain, purport to generate admissibility-relevant governance from a lower basis, or serve as an independent same-domain invariant alongside the kernel. These possibilities cover the relevant ways an additional condition could relate to fixed-domain admissibility: it may change admissibility classification, supply or generate it, or leave it untouched. If it leaves admissibility untouched, then it performs no independent governance work and is derivative rather than foundational.

Theorem 3.79 (Exhaustion of foundational conditions)**.** *Let Q be any purported additional foundational condition. Then exactly one of the following holds:*

(i) Q performs no independent governance work beyond the kernel K, in which case Q is derivative rather than foundational;

(ii) Q purports to generate admissibility-relevant governance from a lower same-domain basis; or

(iii) Q purports to persist as an independent same-domain classifier or invariant alongside the kernel.

Cases (ii) and (iii) are impossible: generation from below is excluded by the non-derivability and no-faithful-lower-generator results, while independent same-domain persistence is excluded by the no-faithful-same-domain-extension results together with the absence of any faithful same-domain counterexample. Therefore no additional foundational condition survives on the same fixed domain as an independent bearer of governance work.

Proof. Take any proposed foundational candidate Q. If Q makes no difference to the class of governed constructions once K is fixed, then by definition it is not foundational; it is descriptive, not load-bearing. Suppose instead that Q performs independent governance work on the same fixed domain. Then, by the classification just stated, that extra work must enter in one of two nontrivial ways: either Q is supposed to generate admissibility-relevant governance from a lower same-domain basis, or it is supposed to survive as an independent same-domain classifier or invariant alongside the kernel.

If Q is claimed to be generated from a lower same-domain basis, then Corollary 3.45 together with Theorem 3.46 excludes the claim: no member of the kernel, and no genuinely foundational governance role, is generated from below while remaining on the same fixed domain.

If Q is instead claimed to persist as an independent same-domain classifier or invariant, then it would constitute an additional same-domain bearer of governance work. But this is excluded by Theorem 3.48: there is no faithful same-domain extension that converts the original acts into a second independent governance architecture rather than collapsing back to the original admissibility boundary, importing illicit structure, violating the base goods, or proving trivial.

A purported fourth option would have to leave admissibility untouched while still doing independent foundational work. But a condition that leaves admissibility untouched leaves the class of governed constructions untouched as well, and so is derivative rather than foundational. Therefore the listed cases are complete. Hence any such condition either does no independent work or falls into one of the two excluded branches. Therefore no additional foundational condition survives as an independent bearer of governance work on the same fixed domain. □

Corollary 3.80 (Foundational closure)**.** *The foundational layer of non-degenerate construction is complete in the following sense: no admissible extension below or alongside K survives on the same fixed domain as an independent bearer of governance work.*

Proof. Immediate from Theorem 3.79. □

3.8 Mechanization Boundary and Proof-Theoretic Scope

Theorem 3.81 (Internal mechanization boundary)**.** *No proof system can establish the package K as a genuinely new internal theorem from a lower layer while preserving the target domain fixed in the fixed-domain construction subsection.*

Proof. Any proof system that already counts its outputs as admissibility-assessable constructions belongs to the target class of the fixed-domain construction subsection. By Theorem 3.31, any such system already operates with admissibility, standing, reference, and irreversibility in place. An "internal construction" of K therefore begins under the very conditions it purports to generate. By Theorems 3.46, 3.47, and 3.48, this is not generation from below but restatement, encoding, or circular recapture. □

Remark 3.82. The point is not anti-formal. It is to mark where formal work properly begins. Once the fixed package is in place, the remaining consequence layer is ordinary derivable mathematics and is an entirely appropriate target for mechanization.

Corollary 3.83 (No lower internal substrate)**.** *A faithful mechanization of the present framework begins at the consequence layer rather than beneath admissibility and standing.*

Proof. If a lower internal layer still counted as part of the same construction regime, then Theorem 3.31 would already place admissibility and standing there. If it did not count as part of that regime, it would not amount to a lower constructional substrate for the present target. So the correct starting point for mechanization is the downstream consequence layer. □

Remark 3.84. A companion formalization is included only as corroboration of this downstream layer. It is not a premise in the argument of the self-contained kernel proof, and the non-derivability results do not depend on repository-level or file-level detail. For present purposes it is enough that representative families—split-collapse, standing/reuse closure, no same-act repair, no generators, no independent carriers, uniqueness of the admissible interior, conservation, and the supporting transport/invariance lemmas—have been machine-checked. That mechanized layer matters here not as a substitute for the present proofs, but as an external check that the equivalence-level uniqueness claims do not rest on merely verbal factorization.

3.9 Endpoint-facing kernel wrappers

Definition 3.85 (Theorem-bearing endpoint use)**.** For a fixed endpoint context R and a branch B,

$$\mathsf{EndpointUse}_R(B)$$

means that B is offered as a theorem-bearing resolution of the official endpoint image of R. Endpoint use is not raw inscription and not mere report language; it is a governed mathematical act with target, carrier, theorem-status, redescription stability, and downstream reuse.

Theorem 3.86 (Endpoint use is governed construction)**.** *For every fixed endpoint context R and branch B,*

$$\mathsf{EndpointUse}_R(B) \Rightarrow \mathrm{Constr}(B).$$

Consequently endpoint use is subject to the kernel of non-degenerate construction.

Proof. A theorem-bearing endpoint must fix the target it resolves, preserve the carrier on which the problem is stated, distinguish theorem-status from raw trace or failed proof, permit downstream mathematical reuse, and remain stable under admissible redescription. These are precisely the features of governed construction for a determinate object on a fixed domain. Therefore endpoint use is governed construction. □

Theorem 3.87 (Kernel necessity for theorem-bearing endpoints)**.** *If* EndpointUse*, then the full kernel*

$$K = \{\mathrm{Adm}, \mathrm{St}, \mathrm{Ref}, \mathrm{Irr}\}$$

is already in force for that endpoint use.

Proof. By Theorem 3.86, endpoint use is governed construction. By Theorem 3.31, every meaningful non-degenerate governed construction already instantiates admissibility, standing, reference, and irreversibility. Thus the kernel is already in force for theorem-bearing endpoint use. □

Theorem 3.88 (Ordinary theoremhood is kernel-internal)**.** *For any proposition P, if* $\mathsf{Proof}_R(P)$ *is a genuine proof rather than raw trace, then*

$$\mathsf{Proof}_R(P) \Rightarrow \mathrm{Der}(P) \Rightarrow K_R.$$

In particular, an ordinary theorem of $\neg\mathsf{Pos}_{\mathsf{raw}}$*, if it exists as a theorem, is not rejected merely because it is negative.*

Proof. A genuine proof is a proposition-targeted governed derivation. By Definition 3.8, derivation is the proposition-targeted special case of governed construction. Corollary 3.38 states that every governed derivation already presupposes the full kernel. Therefore ordinary theoremhood is kernel-internal. The sign or polarity of the proposition does not change this conclusion. □

4 A+ as Kernel-Forced Consequence Layer

Definition 4.1 (A+ endpoint consequence package)**.** For an endpoint context R and branch B, A^+ denotes the fixed-domain consequence package forced by kernel-instantiated theorem-bearing endpoint use. It consists of boundary fixation, endpoint bivalence, AMetric boundary, standing as reuse-stable admissibility, conservation of standing, unique admissible interior, no raw trace sufficiency, no selector import, no carrier/domain transfer, no third endpoint status, no auxiliary endpoint-status refinement, and report-preserving output discipline.

Theorem 4.2 (Kernel forces A+ consequences)**.** *For every fixed endpoint context R and branch B,*

$$\mathsf{EndpointUse}_R(B) \wedge \mathsf{FixedDomain}(R) \Rightarrow \mathsf{A}^+_R(B).$$

Proof. By Theorem 3.87, endpoint use instantiates the kernel. Fixed-domain closure then applies the consequences proved in the kernel section. Boundary fixation is admissibility as the joint governance boundary. Endpoint bivalence follows from Theorem 3.64. The AMetric boundary follows from Theorem 3.65. Standing as reuse-stable admissibility follows from Theorem 3.66. Unique admissible

interior follows from Theorem 3.67. Conservation follows from Theorem 3.68. No raw trace sufficiency follows from Lemma 3.19. No selector import and no carrier/domain transfer follow from Proposition 3.49 and Theorem 3.69. No third endpoint status follows from Lemma 3.63. No auxiliary endpoint-status refinement follows from structural rigidity. Report-preserving output discipline is the claim-report interface of the same reference, standing, admissibility, and irreversibility roles. Thus A+ is a consequence package, not an independent premise. □

Corollary 4.3 (AMetricity from endpoint use)**.** *On the fixed SAT endpoint image,*

$$\mathcal{C}_{\mathsf{SAT}}(R, m) \wedge \mathsf{EndpointUse}_R(B) \wedge \mathsf{FixedDomain}(R) \Rightarrow \mathsf{AMetricBoundary}(R).$$

Proof. The context supplies the carrier and branch slots. Endpoint use plus fixed-domain closure supplies A^+ by Theorem 4.2; the AMetric boundary is one of the listed kernel-forced A+ consequences. □

5 Endpoint-Status Discriminator Discipline

Definition 5.1 (Endpoint-status discriminator)**.** An endpoint-status discriminator for a branch B on context R, written

$$\mathsf{EndpointDisc}_R(C, B),$$

is a status factor, rule, classifier, role-marker, selector, or admissibility-relevant distinction by which B is asserted to occupy an endpoint status on the fixed endpoint image of R.

Definition 5.2 (Independent same-domain discriminator)**.** A discriminator C is independent same-domain governance work for B, written $\mathsf{IndependentDisc}_R(C, B)$, iff it changes endpoint-status, admissibility, standing-bearing continuation, or reportable theorem use on the same fixed domain while not being governance-equivalent to the already fixed admissibility/standing interface.

Theorem 5.3 (Endpoint discriminator trichotomy with domain exit)**.** *Let C be any endpoint-status discriminator attached to branch B on a fixed domain R. Exactly one of the following governance cases applies:*

$$\begin{array}{ll} \mathsf{Bookkeeping}_R(C, B), & \mathsf{GovEquivalent}_R(C, B), \\ \mathsf{IndependentDisc}_R(C, B), & \mathsf{DomainShift}_R(C, B). \end{array}$$

That is, C either changes no endpoint-governance verdict, reproduces the existing kernel-forced interface, performs independent same-domain governance work, or changes the admissibility-relevant invariant bundle and leaves the domain.

Proof. If C changes no admissibility, standing, continuation, endpoint-status, or reportability verdict, it is bookkeeping. If it changes verdicts only in a way already fixed by the kernel-forced admissibility/standing interface, it is governance-equivalent. If it changes endpoint-status verdicts on the same invariant bundle without being governance-equivalent, it performs independent same-domain governance work. If it changes the invariant bundle by which target, carrier, standing-bearing continuation, or admissibility classification are individuated, it changes domain. These cases exhaust whether a discriminator changes governance work and whether the change remains on the fixed domain. □

Theorem 5.4 (Independent same-domain discriminator is forbidden)**.** *For theorem-bearing endpoint use on a fixed domain,*

$$\mathsf{EndpointUse}_R(B) \wedge \mathsf{FixedDomain}(R) \Rightarrow \neg\exists C\, \mathsf{IndependentDisc}_R(C, B).$$

Proof. Endpoint use plus fixed-domain closure yields the A+ consequence package by Theorem 4.2. The AMetric boundary excludes selector, ranking, metric, or parameter work at the endpoint boundary. Bivalence excludes third endpoint status. Standing as reuse-stable admissibility and the unique admissible interior exclude a second same-domain endpoint gate. Conservation excludes creation, recovery, transfer, or amplification of standing by an extra carrier. Transport closure excludes same-domain redescription changes. Therefore no independent same-domain endpoint-status discriminator survives. By Theorem 3.69, any attempted realization is governance-equivalent, illicit same-domain selector work, domain-changing, or bookkeeping; the independent same-domain case is the illicit case. □

Corollary 5.5 (Role-specific classification is consequence, not source)**.** *If a branch has a role-specific endpoint status, that status is admissible only as a consequence of endpoint use under the kernel-forced fixed-domain interface. It does not create standing for the branch.*

Proof. By Theorem 3.87, endpoint use already presupposes the kernel. By Theorem 4.2, the endpoint-status interface is downstream of that kernel. Hence role-specific classification may record the status a branch has under the fixed interface, but cannot be a source of kernel standing. □

6 SAT Branch Asymmetry and Separator Occupation

Definition 6.1 (Ordinary negative theorem)**.**

$$\mathsf{OrdNegThm}_R := \mathsf{Proof}_R(\neg \mathsf{Pos}_{\mathsf{raw}}).$$

This denotes ordinary theoremhood of the negative SAT proposition inside a governed theorem regime. It is not, by itself, a distinct separator endpoint-status discriminator.

Theorem 6.2 (Negative theoremhood is not automatically defective)**.**

$$\mathsf{OrdNegThm}_R \Rightarrow K_R.$$

Thus the manuscript does not reject $\mathsf{Proof}_R(\neg \mathsf{Pos}_{\mathsf{raw}})$ *merely for being negative.*

Proof. This is Theorem 3.88 applied to $P = \neg \mathsf{Pos}_{\mathsf{raw}}$. Genuine theoremhood is governed derivation and is already kernel-internal. The excluded object below is not theoremhood of a negative proposition; it is the same-domain separator discrimination induced by the SAT image-separator branch. □

Theorem 6.3 (Ordinary negative theoremhood is not the rejected discriminator)**.** *Ordinary theoremhood of the negative proposition is not, merely as theoremhood, the independent same-domain separator discriminator excluded below:*

$$\mathsf{OrdNegThm}_R \textit{ by theorem status alone does not equal } \mathcal{D}_{\mathsf{sep}}(\textit{model}).$$

When the content proved is the raw negative branch $\mathsf{Sep}_{\mathsf{bare}}$*, the SAT-specific lower-bound/no-positive bridge transports that content directly to the image-separator branch* $\mathsf{Sep}_{\mathsf{img}}$*; the resulting contradiction targets the discriminator induced by that branch, not negative theoremhood as such.*

Proof. Ordinary theoremhood is a proposition-targeted governed derivation, hence kernel-internal by Theorem 6.2. The discriminator excluded below is not a theorem-status predicate. It is the endpoint-status discriminator induced by the SAT image-separator branch obtained in Theorem 6.12. Thus an ordinary negative theorem is not rejected for polarity or for being theoremhood; the rejected object is the same-domain separator endpoint-status discrimination induced by the raw negative branch through the image-separator bridge. □

Definition 6.4 (Official negative endpoint resolution)**.**

$$\mathsf{OfficialEndpointResolution}_R(\neg\mathsf{Pos}_{\mathsf{raw}})$$

means that an ordinary proof of the negative proposition is offered not merely as a proposition-targeted derivation, but as the official negative resolution of the P versus NP endpoint on the unrestricted finite CNF-SAT carrier. This predicate records use of the proof as an endpoint act. It is not a source of theorem standing and does not make negative theoremhood defective.

Theorem 6.5 (Official negative theoremhood becomes endpoint use)**.** *There is no standing intermediate position between ordinary negative theoremhood and negative endpoint use when the theorem is offered as a resolution of the official P versus NP endpoint. Formally, on the fixed SAT carrier,*

$$\mathcal{C}_{\mathsf{SAT}}(R,m) \wedge \mathsf{FixedDomain}(R) \wedge \mathsf{Model}_{\mathsf{CNF}}(\mathit{model},R) \wedge \mathsf{OrdNegThm}_R \\ \wedge\, \mathsf{OfficialEndpointResolution}_R(\neg\mathsf{Pos}_{\mathsf{raw}}) \Rightarrow \mathsf{EndpointUse}_R(\mathsf{Sep}_{\mathsf{bare}}).$$

Proof. An ordinary proof of $\neg\mathsf{Pos}_{\mathsf{raw}}$, considered only as a proposition-targeted derivation, is governed theoremhood and is not rejected for being negative. But the additional premise $\mathsf{OfficialEndpointResolution}_R(\neg\mathsf{Pos}_{\mathsf{raw}})$ changes the act under assessment: the proof is now being offered as the official negative resolution of the unrestricted finite CNF-SAT endpoint. By Definition 3.85, theorem-bearing endpoint use is precisely the governed mathematical act that fixes the target, carrier, theorem-status, redescription stability, and downstream reuse of a branch. Since the branch offered is the raw negative branch $\mathsf{Sep}_{\mathsf{bare}}$ on the fixed SAT carrier, the act is $\mathsf{EndpointUse}_R(\mathsf{Sep}_{\mathsf{bare}})$. If the proof refuses that endpoint-use status, then it is no longer being used as the official endpoint resolution of the same target; it is merely non-endpoint theoremhood, bookkeeping negation, carrier shift, or a degenerate act relative to the fixed endpoint. □

Corollary 6.6 (No floating official negative endpoint)**.** *A proof of $\neg\mathsf{Pos}_{\mathsf{raw}}$ cannot simultaneously be offered as the official negative resolution of the unrestricted finite CNF-SAT endpoint and refuse negative endpoint-use status. If it is not endpoint use, it does not resolve the official endpoint; if it is endpoint use, it enters the SAT negative-occupation and endpoint-status-bivalence results below.*

Proof. Immediate from Theorem 6.5. The only harmless form of ordinary negative theoremhood is theoremhood considered without official endpoint-resolution use. Once the theorem is offered as the endpoint resolution, refusing the endpoint-use classification withholds the very status required for the act to be the same theorem-bearing resolution act. □

Theorem 6.7 (Positive branch is direct constructive endpoint occupation)**.** *The positive CNF-SAT branch directly occupies the constructive endpoint carrier:*

$$\mathsf{CnfPositiveEndpoint} \Rightarrow \mathsf{CnfSATDirectPositiveEndpointOccupation}.$$

Equivalently, when $\mathsf{Pos}_{\mathsf{raw}}$ is obtained as CnfSATInPolyTime*, its endpoint role is the direct polynomial-time CNF-SAT endpoint. It is not a same-domain separator classifier and does not require an independent endpoint-status discriminator.*

Proof. The positive branch asserts the existence of a deterministic polynomial-time decision procedure for arbitrary finite CNF-SAT. That assertion directly occupies the constructive endpoint carrier named by CnfSATInPolyTime. No further separator relation over the endpoint image is introduced: the branch does not classify the positive route from outside; it is the positive route. The corresponding audited formal theorem is listed in Appendix A. □

Theorem 6.8 (Raw positive branch equals the formal CNF positive endpoint)**.** *On any context-bound CNF-SAT endpoint model,*

$$\mathcal{C}_{\mathsf{SAT}}(R, m) \wedge \mathsf{FixedDomain}(R) \wedge \mathsf{Model}_{\mathsf{CNF}}(\mathit{model}, R) \\ \Rightarrow (\mathsf{Pos}_{\mathsf{raw}} \leftrightarrow \mathsf{CnfPositiveEndpoint}).$$

Proof. The premise $\mathsf{Model}_{\mathsf{CNF}}(\mathit{model}, R)$ fixes the formal CNF-SAT endpoint model associated with the manuscript's raw carrier. By Definition 1.5, $\mathsf{Pos}_{\mathsf{raw}} := \mathsf{CnfSATInPolyTime}$, the assertion that arbitrary finite CNF-SAT is decidable in deterministic polynomial time. In the formal SAT operator layer, CnfPositiveEndpoint names that same positive endpoint on the encoded CNF model. Thus the raw positive branch and the formal positive endpoint are equivalent on the fixed model. □

Corollary 6.9 (Bare negative branch equals the formal bare negative branch)**.** *On any context-bound CNF-SAT endpoint model,*

$$\mathcal{C}_{\mathsf{SAT}}(R, m) \wedge \mathsf{FixedDomain}(R) \wedge \mathsf{Model}_{\mathsf{CNF}}(\mathit{model}, R) \wedge \mathsf{Sep}_{\mathsf{bare}} \\ \Rightarrow \mathsf{CnfSATBareNegativeBranch}(R, \mathit{model}).$$

Proof. By Theorem 7.1, $\mathsf{Sep}_{\mathsf{bare}} \leftrightarrow \neg\mathsf{Pos}_{\mathsf{raw}}$. By Theorem 6.8, $\mathsf{Pos}_{\mathsf{raw}} \leftrightarrow \mathsf{CnfPositiveEndpoint}$ on the context-bound CNF model. Hence $\mathsf{Sep}_{\mathsf{bare}}$ gives $\neg\mathsf{CnfPositiveEndpoint}$, which is exactly $\mathsf{CnfSATBareNegativeBranch}(R, \mathit{model})$ in the formal SAT operator notation. □

Definition 6.10 (Context-bound CNF model)**.**

$$\mathsf{Model}_{\mathsf{CNF}}(\mathit{model}, R)$$

means that *model* is the encoded CNF-SAT endpoint model associated with the fixed SAT context R: the carrier is arbitrary finite CNF over finite Boolean alphabets, the input measure is formula length, and admissible encodings preserve the same endpoint-image carrier.

Theorem 6.11 (Canonical CNF model binding is instantiated, not arbitrary)**.** *The fixed SAT context supplies its own canonical encoded CNF model, denoted m_{SAT}, rather than making an arbitrary model context-bound:*

$$\mathcal{C}_{\mathsf{SAT}}(R, m)_{R^{\mathsf{ctx}}_{\mathsf{SAT}}} \wedge \mathsf{FixedDomain}(R^{\mathsf{ctx}}_{\mathsf{SAT}}) \Rightarrow \mathsf{Model}_{\mathsf{CNF}}(m_{\mathsf{SAT}}, R^{\mathsf{ctx}}_{\mathsf{SAT}}).$$

All branch-local theorems below keep $\mathsf{Model}_{\mathsf{CNF}}(\mathit{model}, R)$ as an explicit premise until the final instantiated SAT context supplies m_{SAT}.

Proof. The CNF model is fixed by the official SAT endpoint carrier: arbitrary finite CNF instances over finite Boolean alphabets, evaluated by formula size, with admissible encodings preserving the same endpoint-image carrier. This determines the canonical context-bound model for $R^{\mathsf{ctx}}_{\mathsf{SAT}}$. It does not license the stronger and false reading that every arbitrary *model* is context-bound. □

Theorem 6.12 (Raw negative branch forces the SAT image-separator branch)**.** *In the fixed SAT endpoint setting, the raw negative branch transports directly to the formal SAT image-separator branch:*

$$\mathcal{C}_{\mathsf{SAT}}(R, m) \wedge \mathsf{FixedDomain}(R) \wedge \mathsf{Model}_{\mathsf{CNF}}(\mathit{model}, R) \wedge \mathsf{Sep}_{\mathsf{bare}} \Rightarrow \mathsf{Sep}_{\mathsf{img}}.$$

Proof. The implication is not a reportability principle and does not assert that raw truth creates a Clay report act. By Corollary 6.9, the manuscript-level assumption $\mathsf{Sep}_{\mathsf{bare}}$ is identified with the formal SAT operator branch $\mathsf{CnfSATBareNegativeBranch}(R, model)$, i.e. with $\neg\mathsf{CnfPositiveEndpoint}$ on the context-bound CNF model. The audited theorem listed in Appendix A proves

$$\mathsf{CnfSATBareNegativeBranch}(R, model) \Rightarrow \mathsf{CnfSATImageSeparatorBranch}(R, model),$$

using the lower-bound/no-positive equivalence

`cnfDirectGateLowerBoundResidualTarget_iff_noSatInPolyTime`.

In this manuscript, $\mathsf{CnfSATImageSeparatorBranch}(R, model)$ is denoted by $\mathsf{Sep}_{\mathsf{img}}$. Thus the raw negative SAT branch is not laundered through reportability, not treated as defective theoremhood, and not routed through an additional residual layer. It is transported directly into the SAT image-separator branch by the formal SAT lower-bound/no-positive bridge. □

6.1 SAT Negative Occupation Exhaustion and Endpoint-Status Bivalence

The preceding bridge is not a representational convention. It is the SAT-local lower-bound/no-positive equivalence on the context-bound unrestricted finite CNF-SAT endpoint model. The current Lean formalization separates two locks that make the negative-occupation step explicit. First, official negative endpoint use is exhausted into four cases: image-separator occupation, ordinary negative alternative, bookkeeping-only negative, or carrier/domain shift. Second, governed endpoint use on the fixed finite CNF-SAT carrier has only two endpoint statuses: positive occupation or separator occupation. These locks are included to block the claim that the official raw negative may remain a third theorem-bearing endpoint status that is neither positive nor separator.

The phrase "official negative endpoint use" is not a new escape hatch and not an AASC-added source of standing. By Theorem 6.5, ordinary theoremhood of $\neg\mathsf{Pos}_{\mathsf{raw}}$ becomes negative endpoint use exactly when it is offered as the official negative resolution of the unrestricted finite CNF-SAT endpoint. If it is not so offered, it does not resolve the official target. If it is so offered, it enters the occupation-exhaustion and bivalence locks in this subsection.

Definition 6.13 (Official negative occupation alternatives)**.** For the context-bound CNF-SAT endpoint model, $\mathsf{CnfSATOfficialNegativeEndpointUse}(R, model)$ denotes theorem-bearing official negative endpoint use of the finite CNF-SAT carrier. The Lean surface distinguishes four exhaustive negative-occupation alternatives:

$$\mathsf{Sep}_{\mathsf{img}}, \qquad \mathsf{OrdNeg}_R^{\mathsf{alt}},$$
$$\mathsf{CnfSATBookkeepingNegativeOnly}(R, model), \qquad \mathsf{CnfSATCarrierShift}(R, model).$$

The first is image-separator occupation. The second is a putative ordinary negative endpoint alternative that remains governed but does not occupy separator status. The third is bookkeeping-only negation, not theorem-bearing endpoint use. The fourth is a change of carrier or domain. The corresponding Lean objects are `CnfSATOfficialNegativeEndpointUse`, `CnfSATNegativeOccupationExhaustion`, `CnfSATBookkeepingNegativeOnly`, and `CnfSATCarrierShift`.

Theorem 6.14 (SAT negative occupation exhaustion)**.** *Official negative endpoint use on the context-bound finite CNF-SAT carrier is exhausted by image-separator occupation, ordinary negative alternative, bookkeeping-only negative, or carrier/domain shift. In Lean this is the theorem* `cnfSATNegativeOccupation_exhaustion`.

Proof. The exhaustion is branch-complete by construction of the fixed SAT endpoint image. A theorem-bearing negative endpoint either occupies the image-separator branch, claims a distinct governed ordinary negative endpoint status, remains a bookkeeping-only negation without endpoint use, or changes the carrier/domain on which the official endpoint is being evaluated. These alternatives cover whether the negative is endpoint-bearing and, if endpoint-bearing, whether it remains on the fixed carrier and whether it occupies separator status. The cited Lean anchor records this exact exhaustion. □

Theorem 6.15 (Non-optionality of SAT negative occupation)**.** *On the fixed context-bound finite CNF-SAT endpoint carrier, official negative endpoint use together with fixed endpoint domain, context-bound CNF model, and the AMetric boundary entails image-separator occupation:*

$$\mathsf{CnfSATOfficialNegativeEndpointUse}(R, model) \wedge \mathsf{FixedDomain}(R) \wedge \mathsf{Model}_{\mathsf{CNF}}(model, R) \wedge \mathsf{AMetricBoundary}(R) \Rightarrow \mathsf{Sep}_{\mathsf{img}}.$$

The Lean anchor is `cnfSATNegativeOccupation_nonoptional`.

Proof. Apply the occupation exhaustion of Theorem 6.14. Fixed endpoint domain excludes carrier/-domain shift; official negative endpoint use excludes bookkeeping-only negation; and the AMetric boundary excludes the ordinary-negative-alternative branch. The corresponding Lean anchors are:

```
cnfSATNoCarrierShift_of_fixedEndpointDomain
cnfSATOfficialNegativeEndpointUse_not_bookkeepingOnly
cnfSATNoOrdinaryNegativeAlternative_of_ametricBoundary
cnfSATNegativeOccupation_nonoptional
```

The remaining exhaustive branch is the SAT image-separator branch. □

Definition 6.16 (SAT endpoint-status bivalence)**.** The Lean status type `CnfSATEndpointStatus` has exactly two constructors, `positive` and `separator`. The predicates `CnfSATGovernedEndpointUse` and `CnfSATGovernedEndpointStatusUse` mark governed endpoint use and governed endpoint-status use on the context-bound finite CNF-SAT carrier.

Theorem 6.17 (Endpoint-status bivalence for governed CNF-SAT endpoint use)**.** *Governed endpoint use on the fixed finite CNF-SAT carrier is endpoint-status bivalent: it has positive status or separator status, and no third governed endpoint status. The Lean anchors are* `cnfSATEndpointStatus_exhaustive` *and* `cnfSATGovernedEndpointUse_bivalent`.

Proof. The endpoint-status surface contains only the positive and separator constructors. A governed endpoint use must occupy one of these statuses. Any alleged third governed endpoint status would either be bookkeeping, a carrier shift, or an independent same-domain status refinement; those alternatives are excluded by the fixed-domain endpoint interface already established in Sections 5 and 6. The Lean anchors record the status-exhaustive and governed-use bivalence forms. □

Theorem 6.18 (Negative governed endpoint use has separator status)**.** *For the governed finite CNF-SAT endpoint,*

$$\mathsf{CnfSATGovernedEndpointUse}(R, model) \wedge \neg\mathsf{CnfPositiveEndpoint} \Rightarrow \mathsf{CnfSATImageSeparatorBranch}(R, model).$$

Equivalently, a governed endpoint use that is not positive occupies separator status and therefore the image-separator branch. The Lean anchors are `cnfSATNegativeGovernedEndpointUse_has_separatorStatus` *and* `cnfSATImageSeparatorBranch_of_negativeGovernedEndpointUse`.

Proof. By Theorem 6.17, governed endpoint use is positive or separator. The assumption $\neg\mathsf{CnfPositiveEndpoint}$ eliminates positive endpoint occupation. The only remaining governed endpoint status is separator status, and separator status is the SAT image-separator branch on the context-bound CNF model. This is the direct branch form formalized by `cnfSATImageSeparatorBranch_of_negativeGovernedEndpointUse`. □

Consequently, the objection that the official raw negative may be theorem-bearing while avoiding image-separator occupation is not a remaining third route. Mere proposition-targeted negative theoremhood is harmless but does not by itself resolve the official endpoint. Once the same theorem is offered as the official negative endpoint resolution, Theorem 6.5 makes it endpoint use; on the fixed governed finite CNF-SAT carrier, a negative endpoint is not positive; endpoint-status bivalence forces separator status; separator status occupies the image-separator branch; and the image-separator branch is the branch closed by the no-independent-discriminator route below.

Definition 6.19 (SAT-local independence bridge)**.** $\mathcal{I}_{\mathsf{sep}}(model)$ denotes the SAT-local same-domain independence bridge: a separating candidate-status classifier for the context-bound CNF-SAT model is independent same-domain endpoint-status discrimination. In the Lean surface this is the predicate $\mathsf{CnfSeparatingClassifierIsIndependentSameDomain}(model)$. It is not a source of standing and not an A+ premise; it records the SAT-local condition under which an image-separator branch produces the independent discriminator excluded by kernel-forced rigidity.

Definition 6.20 (Separator discriminator and no-independent closure)**.** For the context-bound CNF-SAT model, $\mathcal{D}_{\mathsf{sep}}(model)$ denotes the independent same-domain endpoint-status discriminator induced by the SAT image-separator branch:

$$\mathcal{D}_{\mathsf{sep}}(model) := \mathsf{CnfSATIndependentSeparatorEndpointStatusDiscriminator}(model).$$

The predicate $\mathcal{N}_{\mathsf{sep}}(model)$ denotes the kernel-forced no-independent-separator-classifier closure for that same model:

$$\mathcal{N}_{\mathsf{sep}}(model) := \mathsf{CnfNoIndependentSeparatingClassifier}(model).$$

It is used only at the endpoint-use consequence layer, and its operative content is that no independent same-domain separator discriminator survives on the fixed CNF-SAT endpoint carrier.

Theorem 6.21 (No-independent closure excludes the separator discriminator)**.** *For the context-bound CNF-SAT model,*

$$\mathcal{N}_{\mathsf{sep}}(model) \Rightarrow \neg\mathcal{D}_{\mathsf{sep}}(model).$$

Proof. By Definition 6.20, $\mathcal{D}_{\mathsf{sep}}(model)$ is the formal independent separator endpoint-status discriminator and $\mathcal{N}_{\mathsf{sep}}(model)$ is the corresponding no-independent-separator-classifier closure. The latter is precisely the negation of the former at the fixed SAT endpoint carrier: independent same-domain separator discrimination is excluded. Thus $\mathcal{N}_{\mathsf{sep}}(model) \Rightarrow \neg\mathcal{D}_{\mathsf{sep}}(model)$. □

Theorem 6.22 (SAT-local bridge instantiated by the image-separator branch)**.** *In the fixed SAT context, the SAT-local independence bridge is invoked only by the image-separator branch:*

$$\mathcal{C}_{\mathsf{SAT}}(R,m) \wedge \mathsf{FixedDomain}(R) \wedge \mathsf{Model}_{\mathsf{CNF}}(model,R) \wedge \mathsf{Sep}_{\mathsf{img}} \Rightarrow \mathcal{I}_{\mathsf{sep}}(model).$$

Proof. The model-binding clause fixes the encoded CNF-SAT carrier: arbitrary finite CNF instances over finite Boolean alphabets, evaluated by formula size, with admissible encodings preserving the

same SAT endpoint image. When $\mathsf{Sep}_{\mathsf{img}}$ is present, the branch is the SAT image-separator branch on that same carrier. The separating classifier induced by that branch is therefore same-domain rather than transported, external, or report-bookkeeping. This is the SAT-local bridge represented in Lean by $\mathsf{CnfSeparatingClassifierIsIndependentSameDomain}(model)$. The theorem does not assert an independent discriminator from context alone; it states the branch-local condition under which the image-separator theorem applies. □

Theorem 6.23 (Image-separator branch requires independent separator discriminator)**.** *For the fixed SAT endpoint carrier,*

$$\mathcal{C}_{\mathsf{SAT}}(R,m) \wedge \mathsf{FixedDomain}(R) \wedge \mathsf{Model}_{\mathsf{CNF}}(model,R) \wedge \mathcal{I}_{\mathsf{sep}}(model) \wedge \mathsf{Sep}_{\mathsf{img}} \Rightarrow \mathcal{D}_{\mathsf{sep}}(model).$$

Proof. The formal SAT operator theorem states exactly the displayed manuscript implication: a context-bound independent same-domain separating classifier together with the SAT image-separator branch yields the independent separator endpoint-status discriminator. The corresponding Lean anchor is listed in Appendix A. This is the exact branch asymmetry: the positive branch is direct endpoint occupation, while the image-separator branch induces independent same-domain endpoint-status discrimination. □

Theorem 6.24 (Bare negative branch requires independent separator discriminator)**.** *For the fixed SAT endpoint carrier,*

$$\mathcal{C}_{\mathsf{SAT}}(R,m) \wedge \mathsf{FixedDomain}(R) \wedge \mathsf{Model}_{\mathsf{CNF}}(model,R) \wedge \mathsf{Sep}_{\mathsf{bare}} \Rightarrow \mathcal{D}_{\mathsf{sep}}(model).$$

Proof. By Theorem 6.12, $\mathsf{Sep}_{\mathsf{bare}}$ gives $\mathsf{Sep}_{\mathsf{img}}$. The official-negative-theoremhood lock of Theorem 6.5 blocks the attempted escape in which an ordinary proof of $\neg\mathsf{Pos}_{\mathsf{raw}}$ is offered as the official endpoint resolution while refusing endpoint-use status. Once it is endpoint use, the occupation-exhaustion and endpoint-status-bivalence locks in Theorems 6.15–6.18 make explicit that this is not an optional representation choice: a governed official negative endpoint on the fixed CNF-SAT carrier cannot remain a third status. The branch-local bridge theorem then gives $\mathcal{I}_{\mathsf{sep}}(model)$ from that same $\mathsf{Sep}_{\mathsf{img}}$. Theorem 6.23 gives $\mathcal{D}_{\mathsf{sep}}(model)$. The direct audited formal counterpart is listed in Appendix A. □

Theorem 6.25 (Image-separator branch is endpoint use)**.** *In the fixed SAT context,*

$$\mathcal{C}_{\mathsf{SAT}}(R,m) \wedge \mathsf{FixedDomain}(R) \wedge \mathsf{Model}_{\mathsf{CNF}}(model,R) \wedge \mathsf{Sep}_{\mathsf{img}} \Rightarrow \mathsf{EndpointUse}_R(\mathsf{Sep}_{\mathsf{img}}).$$

Proof. The symbol $\mathsf{Sep}_{\mathsf{img}}$ denotes the formal SAT image-separator branch, not raw syntax and not a reportability act. In formal notation,

$$\mathsf{Sep}_{\mathsf{img}} \equiv \mathsf{CnfSATImageSeparatorBranch}(R,model).$$

In the endpoint-image layer, that branch is the separator branch occupying the SAT endpoint image. Therefore, under the fixed carrier and bound CNF model, its presence is theorem-bearing endpoint use of the image-separator branch. □

Theorem 6.26 (Branch-local no independent SAT separator classifier)**.** *For the fixed SAT endpoint carrier, image-separator endpoint use triggers the kernel-forced no-independent-discriminator consequence:*

$$\mathcal{C}_{\mathsf{SAT}}(R,m) \wedge \mathsf{FixedDomain}(R) \wedge \mathsf{Model}_{\mathsf{CNF}}(model,R) \wedge \mathsf{EndpointUse}_R(\mathsf{Sep}_{\mathsf{img}}) \Rightarrow \mathcal{N}_{\mathsf{sep}}(model).$$

Proof. Endpoint use is governed construction, hence it instantiates the kernel by Theorem 3.87. The fixed-domain A+ consequence layer then supplies the AMetric boundary, unique admissible interior, standing conservation, no-selector discipline, transport closure, and structural rigidity. On a fixed SAT endpoint carrier, an endpoint-status discriminator must be bookkeeping, governance-equivalent, domain-changing, or independent same-domain governance work by Theorem 5.3. The first two cases do not yield $\mathcal{D}_{\mathsf{sep}}(model)$, the third is not the same SAT endpoint domain, and the fourth is excluded by the kernel-forced consequence layer. Thus image-separator endpoint use on the context-bound SAT model yields $\mathcal{N}_{\mathsf{sep}}(model)$. □

Theorem 6.27 (Image-separator branch impossible)**.** *In the fixed SAT context,*

$$\mathcal{C}_{\mathsf{SAT}}(R,m) \wedge \mathsf{FixedDomain}(R) \wedge \mathsf{Model}_{\mathsf{CNF}}(model,R) \Rightarrow \neg \mathsf{Sep}_{\mathsf{img}}.$$

Proof. Assume $\mathcal{C}_{\mathsf{SAT}}(R,m) \wedge \mathsf{FixedDomain}(R) \wedge \mathsf{Model}_{\mathsf{CNF}}(model,R)$ and suppose $\mathsf{Sep}_{\mathsf{img}}$. The branch-local bridge theorem gives $\mathcal{I}_{\mathsf{sep}}(model)$. Theorem 6.25 gives $\mathsf{EndpointUse}_R(\mathsf{Sep}_{\mathsf{img}})$, and Theorem 6.26 gives $\mathcal{N}_{\mathsf{sep}}(model)$. By Theorem 6.21, $\neg\mathcal{D}_{\mathsf{sep}}(model)$. But Theorem 6.23 gives $\mathcal{D}_{\mathsf{sep}}(model)$. Contradiction. Therefore $\neg\mathsf{Sep}_{\mathsf{img}}$. □

No fifth negative occupation. Thus the alleged third governed negative occupation is not a fifth case. If it has no endpoint-governance effect, it is bookkeeping. If it has endpoint-governance effect on the fixed carrier while being neither positive nor separator, it is hidden selector work or forbidden third-status refinement. If it depends on later reclassification of the same act, it is repair-sensitive middle status. If it avoids those consequences only by changing the admissibility-relevant invariant bundle, it is carrier or domain shift. Hence every purported governed negative endpoint occupation that is not separator either fails to be endpoint-bearing, violates the fixed-domain endpoint-status discipline, reopens repair, or leaves the fixed CNF-SAT carrier. No coherent fifth occupation remains.

Theorem 6.28 (Bare negative branch impossible)**.** *In the fixed SAT context,*

$$\mathcal{C}_{\mathsf{SAT}}(R,m) \wedge \mathsf{FixedDomain}(R) \wedge \mathsf{Model}_{\mathsf{CNF}}(model,R) \Rightarrow \neg \mathsf{Sep}_{\mathsf{bare}}.$$

Proof. Assume $\mathcal{C}_{\mathsf{SAT}}(R,m) \wedge \mathsf{FixedDomain}(R) \wedge \mathsf{Model}_{\mathsf{CNF}}(model,R)$ and suppose $\mathsf{Sep}_{\mathsf{bare}}$. By Theorem 6.12, $\mathsf{Sep}_{\mathsf{img}}$. But Theorem 6.27 excludes $\mathsf{Sep}_{\mathsf{img}}$ under the same fixed context. Contradiction. Equivalently, the direct audited formal theorem listed in Appendix A gives the same exclusion of $\mathsf{CnfSATBareNegativeBranch}(R, model)$. □

7 Endpoint-Image Resolution

Theorem 7.1 (Bare SAT endpoint complementarity)**.** *At the raw unrestricted finite CNF-SAT layer,*

$$\mathsf{Sep}_{\mathsf{bare}} \leftrightarrow \neg \mathsf{Pos}_{\mathsf{raw}}.$$

Proof. By Definition 1.5, $\mathsf{Pos}_{\mathsf{raw}} := \mathsf{CnfSATInPolyTime}$, the assertion that arbitrary finite CNF-SAT is decidable by a deterministic polynomial-time machine. The bare negative branch $\mathsf{Sep}_{\mathsf{bare}}$ is the official raw complement branch: failure of polynomial-time decidability for that same unrestricted finite CNF-SAT carrier. Definition 2.1 prevents any shift to a restricted CNF fragment. Therefore the raw branches are exact complements on the same carrier. □

Theorem 7.2 (Bare endpoint bivalence)**.**

$$\mathsf{Pos}_{\mathsf{raw}} \vee \mathsf{Sep}_{\mathsf{bare}}.$$

Proof. Classical bivalence gives $\mathsf{Pos}_{\mathsf{raw}} \vee \neg\mathsf{Pos}_{\mathsf{raw}}$. By Theorem 7.1, $\neg\mathsf{Pos}_{\mathsf{raw}}$ is equivalent to $\mathsf{Sep}_{\mathsf{bare}}$. Hence $\mathsf{Pos}_{\mathsf{raw}} \vee \mathsf{Sep}_{\mathsf{bare}}$. □

Theorem 7.3 (Positive SAT endpoint)**.** *In the fixed SAT context,*

$$\mathcal{C}_{\mathsf{SAT}}(R, m) \wedge \mathsf{FixedDomain}(R) \wedge \mathsf{Model}_{\mathsf{CNF}}(\mathit{model}, R) \Rightarrow \mathsf{Pos}_{\mathsf{raw}}.$$

Proof. Assume $\mathcal{C}_{\mathsf{SAT}}(R, m) \wedge \mathsf{FixedDomain}(R) \wedge \mathsf{Model}_{\mathsf{CNF}}(\mathit{model}, R)$. By Theorem 7.2, either $\mathsf{Pos}_{\mathsf{raw}}$ or $\mathsf{Sep}_{\mathsf{bare}}$. If $\mathsf{Pos}_{\mathsf{raw}}$, the conclusion is immediate. If $\mathsf{Sep}_{\mathsf{bare}}$, Theorem 6.28 contradicts $\mathsf{Sep}_{\mathsf{bare}}$. Therefore $\mathsf{Pos}_{\mathsf{raw}}$. □

Theorem 7.4 (Instantiated SAT endpoint context)**.** *The official unrestricted finite CNF-SAT endpoint carrier instantiates the fixed context, fixed-domain, and bound-model hypotheses:*

$$\mathcal{C}_{\mathsf{SAT}}(R, m)_{R^{\mathsf{ctx}}_{\mathsf{SAT}}} \wedge \mathsf{FixedDomain}(R^{\mathsf{ctx}}_{\mathsf{SAT}}) \wedge \mathsf{Model}_{\mathsf{CNF}}(m_{\mathsf{SAT}}, R^{\mathsf{ctx}}_{\mathsf{SAT}}).$$

Proof. The official SAT endpoint carrier is arbitrary finite CNF over finite Boolean alphabets with formula length as input size; the positive branch is deterministic polynomial-time CNF-SAT decidability; the raw negative branch is its complement on the same carrier; and admissible encodings preserve the carrier and endpoint image. This is exactly the fixed context of Definition 1.3 together with fixed-domain preservation and the context-bound CNF model of Definition 6.10. □

Theorem 7.5 (Instantiated positive SAT endpoint)**.**

$$\mathsf{CnfSATInPolyTime}.$$

Proof. By Theorem 7.4, the official SAT carrier supplies $\mathcal{C}_{\mathsf{SAT}}(R, m)_{R^{\mathsf{ctx}}_{\mathsf{SAT}}} \wedge \mathsf{FixedDomain}(R^{\mathsf{ctx}}_{\mathsf{SAT}}) \wedge \mathsf{Model}_{\mathsf{CNF}}(m_{\mathsf{SAT}}, R^{\mathsf{ctx}}_{\mathsf{SAT}})$. Applying Theorem 7.3 with $R = R^{\mathsf{ctx}}_{\mathsf{SAT}}$ and $\mathit{model} = m_{\mathsf{SAT}}$ yields $\mathsf{Pos}_{\mathsf{raw}}$. By Definition 1.5, $\mathsf{Pos}_{\mathsf{raw}} := \mathsf{CnfSATInPolyTime}$. □

8 Positive Report Classification

Definition 8.1 (Positive Clay report act)**.** $\mathsf{Report}_{\mathsf{Clay}}(\mathsf{Pos}_{\mathsf{raw}})$ is the report act that presents $\mathsf{Pos}_{\mathsf{raw}}$ as the positive mathematical endpoint after the separator branch has been excluded.

Theorem 8.2 (Positive report act)**.**

$$\mathcal{C}_{\mathsf{SAT}}(R, m) \wedge \mathsf{FixedDomain}(R) \wedge \mathsf{Model}_{\mathsf{CNF}}(\mathit{model}, R) \wedge \mathsf{Pos}_{\mathsf{raw}} \Rightarrow \mathsf{Report}_{\mathsf{Clay}}(\mathsf{Pos}_{\mathsf{raw}}).$$

Proof. Once the fixed SAT context is in force and the raw positive endpoint has been derived after exclusion of the separator branch, the manuscript's theorem-bearing output is the report of $\mathsf{Pos}_{\mathsf{raw}}$ as the positive mathematical endpoint. That report act does not supply standing; it records the already derived endpoint under the report-preserving interface forced by the kernel. □

Theorem 8.3 (Positive endpoint classification)**.**

$$\mathcal{C}_{\mathsf{SAT}}(R, m) \wedge \mathsf{FixedDomain}(R) \wedge \mathsf{Model}_{\mathsf{CNF}}(\mathit{model}, R) \wedge \mathsf{Pos}_{\mathsf{raw}} \wedge \mathsf{Report}_{\mathsf{Clay}}(\mathsf{Pos}_{\mathsf{raw}}) \Rightarrow \mathsf{A}_{\mathsf{pos}}.$$

Proof. The positive branch is a theorem-bearing endpoint report on the fixed SAT carrier. By Theorem 6.7, the positive branch is direct constructive endpoint occupation rather than separator classification. By Theorem 3.87, endpoint use is kernel-instantiated; by Theorem 4.2, the report-preserving A+ consequence layer applies. The positive classification $\mathsf{A}_{\mathsf{pos}}$ records that downstream status. It is not a source of standing and is not an independent separator discriminator. □

Corollary 8.4 (Positive Clay mathematical endpoint)**.**

$$\mathcal{C}_{\mathsf{SAT}}(R, m) \wedge \mathsf{FixedDomain}(R) \wedge \mathsf{Model}_{\mathsf{CNF}}(model, R) \Rightarrow \mathsf{Pos}_{\mathsf{Clay}}.$$

Proof. Theorem 7.3 gives $\mathsf{Pos}_{\mathsf{raw}}$. Theorem 8.2 gives $\mathsf{Report}_{\mathsf{Clay}}(\mathsf{Pos}_{\mathsf{raw}})$. Theorem 8.3 gives $\mathsf{A}_{\mathsf{pos}}$. Therefore $\mathsf{Pos}_{\mathsf{Clay}} := \mathsf{Pos}_{\mathsf{raw}} \wedge \mathsf{A}_{\mathsf{pos}}$. □

9 AASC Closure and Endpoint Correspondence

The manuscript does not redefine P, NP, CNF, satisfiability, polynomial time, reduction, or NP-completeness. AASC proves the positive raw SAT endpoint by excluding the separator endpoint image through kernel-forced fixed-domain discriminator closure. Once CnfSATInPolyTime is obtained, the final move to $P = NP$ is the standard Cook–Levin/Karp endpoint correspondence. Cook–Levin/Karp is not the branch-exclusion method; it identifies the official endpoint consequence after the AASC proof has delivered unrestricted finite CNF-SAT in deterministic polynomial time.

Theorem 9.1 (Cook–Levin/Karp endpoint correspondence)**.**

$$\mathsf{CnfSATInPolyTime} \Rightarrow \mathsf{P} = \mathsf{NP}.$$

Proof. Arbitrary finite CNF-SAT is NP-complete under the standard Cook–Levin/Karp polynomial-time many-one reduction framework. If CNF-SAT has a deterministic polynomial-time decider, then every language in NP is decided in deterministic polynomial time by composing its polynomial-time reduction to CNF-SAT with that decider. Hence $\mathsf{NP} \subseteq \mathsf{P}$. Since $\mathsf{P} \subseteq \mathsf{NP}$ by definition, $\mathsf{P} = \mathsf{NP}$. □

Corollary 9.2 (Positive P versus NP endpoint)**.**

$$\mathsf{P} = \mathsf{NP}.$$

Proof. Theorem 7.5 gives CnfSATInPolyTime. Theorem 9.1 gives $\mathsf{P} = \mathsf{NP}$. □

10 Denial Burden and Exhaustion of Remaining Routes

Dependency-inversion objections. An objection that treats the kernel as an optional AASC overlay is not a separate denial route. It is a dependency-inversion objection. Such an objection has standing only if it identifies a failure in the kernel-neutral adequacy argument: either target determinacy, step evaluability, act-time finality, or same-regime fidelity must be shown not to be required for theorem-bearing endpoint use, or a same-regime faithful counterexample must be supplied that preserves all four while omitting at least one kernel role. Without that showing, the objection merely renames a forced precondition as an optional framework and does not reach the proof.

Cost of abandoning the kernel

A denial of the kernel is not a harmless refusal of terminology. The kernel is the role-decomposition of the four adequacy conditions required for determinate same-domain theorem-bearing construction. Accordingly, to deny the kernel while still claiming to assess the same endpoint, an objector must identify which adequacy condition is being abandoned and accept the corresponding loss.

Denied adequacy condition	Kernel role lost	Consequence of denial
Target determinacy	Reference	No fixed endpoint object is preserved.
Step evaluability	Standing	No transition has theorem-bearing stephood.
Act-time finality	Irreversibility	Same-act repair and retrospective status change remain open.
Same-regime fidelity	Admissibility boundary	Carrier, domain, or admissibility classification may shift under redescription.

These are not four optional rules imposed by the present formalism. They are the conditions under which a purported proof, counterproof, endpoint resolution, or theorem-bearing act remains assessable as the same determinate mathematical act. If target determinacy is denied, the objection no longer has the official endpoint as its fixed object. If step evaluability is denied, the objection no longer supplies theorem-bearing construction steps. If act-time finality is denied, the objection permits retrospective repair of the same act and loses stable proof status. If same-regime fidelity is denied, the objection no longer preserves the same carrier or domain under admissible redescription.

Thus a kernel denial has only two possible forms. Either it denies none of the four adequacy conditions, in which case the kernel has already been conceded under different vocabulary; or it denies at least one of them, in which case the objector is no longer preserving determinate same-domain theorem-bearing endpoint use. There is no third position from which one may reject the kernel while retaining the same official endpoint-resolution act.

Consequently, objections that merely say that the kernel is "framework-dependent," "nonstandard," or "not part of ordinary complexity theory" do not yet reach the proof. Such objections must be converted into a precise denial of target determinacy, step evaluability, act-time finality, same-regime fidelity, or a downstream theorem that follows from them. Without that conversion, they are dependency-inversion objections rather than theorem-level objections.

A standing mathematical objection must attack one of the following load-bearing AASC endpoint or endpoint-correspondence claims.

1. A theorem-bearing endpoint can preserve fixed target, fixed carrier, theorem-status distinction, downstream reuse, redescription stability, and act-time finality while lacking one of $\mathrm{Adm}, \mathrm{St}, \mathrm{Ref}, \mathrm{Irr}$.
2. Some A+ component used here is not forced by the kernel plus fixed-domain closure.
3. The kernel or its AASC consequence layer can be treated as an optional overlay that may be accepted or rejected independently of the minimal evaluability conditions for determinate same-domain construction.
4. Ordinary theoremhood is not kernel-internal.
5. Ordinary theoremhood of $\neg\mathsf{Pos}_{\mathsf{raw}}$ can be offered as the official negative endpoint resolution while refusing endpoint-use status, contrary to Theorem 6.5 and Corollary 6.6.
6. The raw negative SAT branch can avoid the SAT image-separator branch despite the SAT lower-bound/no-positive equivalence, contrary to Theorem 6.12.
7. Official negative endpoint use can remain theorem-bearing while avoiding the occupation-exhaustion and endpoint-status-bivalence locks of Theorems 6.15 and 6.18.
8. The positive CNF-SAT branch is not direct constructive endpoint occupation.
9. The SAT image-separator branch can avoid inducing independent same-domain endpoint-status discrimination under the SAT-local independence bridge.
10. The image-separator branch does not instantiate the SAT-local independence bridge on the fixed context-bound CNF model, or the image-separator branch plus that branch-local bridge does not yield $\mathcal{D}_{\mathsf{sep}}(\mathit{model})$.

11. An independent same-domain endpoint-status discriminator can survive structural rigidity, AMetricity, unique interior, conservation, and transport closure.
12. $\mathsf{Sep}_{\mathsf{bare}} \not\Rightarrow \neg\mathsf{Pos}_{\mathsf{raw}}$ at the raw unrestricted finite-CNF layer.
13. $\mathsf{CnfSATInPolyTime}$ does not imply $P = NP$ under the standard Cook–Levin/Karp endpoint correspondence.

Objections outside these classes are non-standing unless they identify a theorem-level failure in one of the cited bridges. The relevant question is whether the manuscript reaches the official mathematical endpoint. Treating AASC as an external overlay is not a separate denial route; it must be converted into a failure of the kernel-neutral adequacy argument or of a stated downstream consequence theorem. It is also not a standing objection to say that a negative theorem would be kernel-defective; the manuscript expressly denies that. The remaining fork is exact: negative theoremhood considered only as theoremhood is harmless but does not resolve the official endpoint; negative theoremhood offered as the official endpoint resolution is endpoint use and enters the SAT negative-occupation lock; a claimed third option is degenerate relative to the fixed endpoint because it withholds the endpoint-use status required for the act to be the same theorem-bearing resolution act. The eliminated negative route is the raw SAT lower-bound/no-positive branch, because that branch transports directly to the SAT image-separator branch; the image-separator branch then induces independent same-domain endpoint-status discrimination under the SAT-local bridge, and the kernel-forced consequence layer excludes such discrimination.

11 Conclusion

The proof has been arranged so that AASC owns the branch-exclusion argument: the kernel is the source of necessity, and AASC is the downstream fixed-domain consequence interface. Raw traces do not become constructions merely by being written. Genuine theoremhood is already governed derivation and is therefore kernel-internal. The positive SAT branch is direct constructive endpoint occupation. The negative SAT branch is not rejected for being negative; ordinary negative theoremhood is harmless as theoremhood. But ordinary negative theoremhood offered as the official P versus NP endpoint resolution is endpoint use of the negative branch. If it refuses endpoint use, it is not the same theorem-bearing resolution act for the official target; if it accepts endpoint use, the SAT occupation-exhaustion and endpoint-status-bivalence locks apply. The raw negative branch then transports directly to the image-separator branch. The image-separator branch invokes the branch-local SAT independence bridge and induces an independent same-domain endpoint-status discriminator. The kernel-forced consequence layer excludes such discriminators. Bare complementarity then yields the positive unrestricted finite CNF-SAT endpoint. The Cook–Levin/Karp endpoint correspondence then identifies this AASC-derived endpoint as $\mathsf{P} = \mathsf{NP}$.

A Lean4 SAT Operator Bridge Appendix

The manuscript proof above remains the primary proof. The Lean4 material in this appendix is support for the local SAT-operator bridge used by the AASC endpoint proof spine:

$$\mathsf{Sep}_{\mathsf{bare}} \Rightarrow \mathsf{Sep}_{\mathsf{img}} \Rightarrow \mathcal{D}_{\mathsf{sep}}(model) \Rightarrow \bot.$$

It is not used as a substitute for the kernel proof, and it does not mechanize the external Cook–Levin/Karp endpoint correspondence from $\mathsf{CnfSATInPolyTime}$ to $P = NP$. Its role is reproducibility support for the SAT endpoint-operator bridge and negative-branch closure formalized in the release archive [9, 10].

A.1 Release, environment, and audit runner

The Lean audit archive included with this project is the following source-material zip:

```
source_material/AASC-PvsNP-SAT-Endpoint-Lean-Audit-1.0.0.zip
```

The pinned build data recorded in the release are:

Item	Value
Lean toolchain	`leanprover/lean4:v4.28.0`
Lake project	`AASCPvsNPSATEndpointLeanAudit`
mathlib requirement	`mathlib`, revision tag `v4.28.0`
Pinned mathlib revision	`8f9d9cff6bd728b17a24e163c9402775d9e6a365`
Main proof queue	`MaleyLean/Papers/PvsNP/SATOperatorProofQueue.lean`
Focused audit runner	`scripts/check-pvsnp-sat-operator-bridge-audit.ps1`
Release root suffix	`062a2a6`

The verification command supplied by the release is:

```
powershell -ExecutionPolicy Bypass -File scripts/check-pvsnp-sat-operator-bridge-audit.ps1
```

The runner prints the Lean toolchain, prints the pinned mathlib manifest revision, builds `MaleyLean.Papers.PvsNP.SATOperatorAuditRunners`, evaluates the SAT-operator formalization-status and progress summaries, scans the audited P vs NP SAT endpoint surface for live `axiom`, `sorry`, `admit`, or `unsafe` declarations, builds the SAT proof queue and bridge-callability modules, and runs the seven focused P vs NP SAT endpoint axiom-check files. The release notes record complete closure for the AASC SAT endpoint and CNF-SAT-in-polynomial-time status summaries. Focused axiom output for the bridge anchors is reported against only the standard Lean/classical/mathlib base dependencies `propext`, `Classical.choice`, and `Quot.sound`.

A.2 Declaration locations

The following declaration locations are taken from the released Lean files. They are included to make the manuscript-to-Lean correspondence checkable without treating Lean as a replacement for the manuscript proof.

```
SameDomainEndpoint.lean:22
  abbrev CnfPositiveEndpoint : Prop := CnfSATInPolyTime

FoundationalClassifierBridge.lean:44
  def CnfSeparatingClassifierIsIndependentSameDomain

FoundationalClassifierBridge.lean:70
  def CnfNoIndependentSeparatingClassifier

SATOperatorProofQueue.lean:10031
  def CnfSATImageSeparatorBranch

SATOperatorProofQueue.lean:10042
  def CnfSATBareNegativeBranch

SATOperatorProofQueue.lean:10049
  def CnfSATBareSeparator

SATOperatorProofQueue.lean:10142
  def CnfSATImageSeparatorEndpointUse

SATOperatorProofQueue.lean:10161
```

```
  def CnfSATDirectPositiveEndpointOccupation

SATOperatorProofQueue.lean:10169
  def CnfSATIndependentSeparatorEndpointStatusDiscriminator

SATOperatorProofQueue.lean:10184
  def CnfSATBookkeepingNegativeOnly
  def CnfSATCarrierShift
  def CnfSATOfficialNegativeEndpointUse
  def CnfSATNegativeOccupationExhaustion

SATOperatorProofQueue.lean:10313
  inductive CnfSATEndpointStatus
  def CnfSATEndpointStatusOccupation
  def CnfSATGovernedEndpointUse
  def CnfSATGovernedEndpointStatusUse
```

The proof-spine theorem locations are:

```
SATOperatorProofQueue.lean:10075
  cnfBareNegativeBranch_forces_imageSeparatorBranch

SATOperatorProofQueue.lean:10184
  cnfSATNegativeOccupation_exhaustion
  cnfSATNoCarrierShift_of_fixedEndpointDomain
  cnfSATOfficialNegativeEndpointUse_not_bookkeepingOnly
  cnfSATNoOrdinaryNegativeAlternative_of_ametricBoundary
  cnfSATNegativeOccupation_nonoptional

SATOperatorProofQueue.lean:10313
  cnfSATEndpointStatus_exhaustive
  cnfSATGovernedEndpointUse_bivalent
  cnfSATNegativeGovernedEndpointUse_has_separatorStatus
  cnfSATGovernedEndpointStatusUse_eq_separator_of_not_positive
  cnfSATImageSeparatorBranch_of_negativeGovernedEndpointUse

SATOperatorProofQueue.lean:10087
  cnfImageSeparatorBranch_of_not_positiveEndpoint

SATOperatorProofQueue.lean:10104
  cnfSATBareSeparator_forces_imageSeparatorBranch

SATOperatorProofQueue.lean:10148
  cnfSATImageSeparatorBranch_endpointUse

SATOperatorProofQueue.lean:10179
  cnfPositiveEndpoint_directEndpointOccupation

SATOperatorProofQueue.lean:10190
  cnfSATImageSeparatorBranch_requires_independentSeparatorDiscriminator

SATOperatorProofQueue.lean:10212
  cnfBareNegativeBranch_requires_independentSeparatorDiscriminator

SATOperatorProofQueue.lean:10246
  cnfSATImageSeparatorBranch_impossible_of_noIndependentDiscriminator

SATOperatorProofQueue.lean:10267
  cnfBareNegativeBranch_impossible_of_noIndependentDiscriminator

SATOperatorProofQueue.lean:10286
  cnfNotPositiveEndpoint_impossible_of_noIndependentDiscriminator

SATOperatorProofQueue.lean:10308
  cnfPositiveEndpoint_of_noIndependentDiscriminator

SATOperatorProofQueue.lean:10331
  cnfSATInPolyTime_of_noIndependentDiscriminator
```

```
SATOperatorProofQueue.lean:10351
  cnfSATInPolyTime_of_context_noIndependentDiscriminator

SATOperatorProofQueue.lean:10372
  cnfSATBareSeparator_impossible_of_context_noIndependentDiscriminator
```

A.3 Manuscript-to-Lean correspondence

The manuscript-to-Lean symbol correspondence is:

```
PosRaw          <-> CnfPositiveEndpoint  (abbrev. CnfSATInPolyTime)
SepBare         <-> CnfSATBareNegativeBranch R model
SepImg          <-> CnfSATImageSeparatorBranch R model
DirPos          <-> CnfSATDirectPositiveEndpointOccupation
SepDisc(model)  <-> CnfSATIndependentSeparatorEndpointStatusDiscriminator model
NoSepDisc(model)<-> CnfNoIndependentSeparatingClassifier model
SepIndBridge(model)
                <-> CnfSeparatingClassifierIsIndependentSameDomain model
OfficialNegUse  <-> CnfSATOfficialNegativeEndpointUse R model
NegOccExhaust   <-> CnfSATNegativeOccupationExhaustion R model
GovernedUse     <-> CnfSATGovernedEndpointUse R model
EndpointStatus  <-> CnfSATEndpointStatus
```

The formal correspondence of the central manuscript chain is:

```
CnfPositiveEndpoint
  -> CnfSATDirectPositiveEndpointOccupation

CnfSATBareNegativeBranch R model
  -> CnfSATImageSeparatorBranch R model

CnfSATOfficialNegativeEndpointUse R model
  + CnfSATFixedEndpointDomain R
  + CnfSATContextBoundCNFModel R model
  + AMetricBoundary R
  -> CnfSATImageSeparatorBranch R model

CnfSATGovernedEndpointUse R model
  + Not CnfPositiveEndpoint
  -> CnfSATImageSeparatorBranch R model

CnfSeparatingClassifierIsIndependentSameDomain model
  -> CnfSATImageSeparatorBranch R model
  -> CnfSATIndependentSeparatorEndpointStatusDiscriminator model

CnfNoIndependentSeparatingClassifier model
  -> CnfSeparatingClassifierIsIndependentSameDomain model
  -> Not (CnfSATImageSeparatorBranch R model)

CnfNoIndependentSeparatingClassifier model
  -> CnfSeparatingClassifierIsIndependentSameDomain model
  -> Not (CnfSATBareNegativeBranch R model)

CnfNoIndependentSeparatingClassifier model
  -> CnfSeparatingClassifierIsIndependentSameDomain model
  -> CnfSATInPolyTime
```

A.4 Selected source excerpts

The following excerpts are reproduced from `SATOperatorProofQueue.lean`. They are the Lean-side anchors for the raw-negative bridge, endpoint-use marker, branch asymmetry, and final SAT endpoint closeout.

```
def CnfSATImageSeparatorBranch
    {Act Object : Type}
    (_R : MinimalConditionsForAdmissibleConstruction.ConstructionRegime Act Object)
    (_model : CnfEncodedCandidateModel) : Prop :=
  cnfDirectGateLowerBoundResidualTarget

def CnfSATBareNegativeBranch
    {Act Object : Type}
    (_R : MinimalConditionsForAdmissibleConstruction.ConstructionRegime Act Object)
    (_model : CnfEncodedCandidateModel) : Prop :=
  Not CnfPositiveEndpoint

theorem cnfBareNegativeBranch_forces_imageSeparatorBranch
    {Act Object : Type}
    {R : MinimalConditionsForAdmissibleConstruction.ConstructionRegime Act Object}
    {model : CnfEncodedCandidateModel}
    (hBare : CnfSATBareNegativeBranch R model) :
    CnfSATImageSeparatorBranch R model :=
  (cnfDirectGateLowerBoundResidualTarget_iff_noSatInPolyTime).2 hBare

theorem cnfImageSeparatorBranch_of_not_positiveEndpoint
    {Act Object : Type}
    {R : MinimalConditionsForAdmissibleConstruction.ConstructionRegime Act Object}
    {model : CnfEncodedCandidateModel}
    (hNotPositive : Not CnfPositiveEndpoint) :
    CnfSATImageSeparatorBranch R model :=
  cnfBareNegativeBranch_forces_imageSeparatorBranch
    (R := R)
    (model := model)
    hNotPositive
```

```
def CnfSATDirectPositiveEndpointOccupation : Prop :=
  CnfPositiveEndpoint

def CnfSATIndependentSeparatorEndpointStatusDiscriminator
    (model : CnfEncodedCandidateModel) : Prop :=
  exists classifier : CnfCandidateStatusClassifier model,
    CnfClassifierSeparatesPolynomialCandidates model classifier /\
      Nonempty (CnfClassifierIndependentSameDomain model classifier)

theorem cnfPositiveEndpoint_directEndpointOccupation
    (hPositive : CnfPositiveEndpoint) :
    CnfSATDirectPositiveEndpointOccupation :=
  hPositive

theorem cnfSATImageSeparatorBranch_requires_independentSeparatorDiscriminator
    {Act Object : Type}
    {R : MinimalConditionsForAdmissibleConstruction.ConstructionRegime Act Object}
    {model : CnfEncodedCandidateModel}
    (hIndependent :
      CnfSeparatingClassifierIsIndependentSameDomain model)
    (hSep : CnfSATImageSeparatorBranch R model) :
    CnfSATIndependentSeparatorEndpointStatusDiscriminator model := by
  have hSameDomain : CnfSameDomainSeparator :=
    (cnfDirectGateLowerBoundResidualTarget_iff_sameDomainSeparator).1 hSep
  let classifier := cnfClassifierOfSameDomainSeparator model hSameDomain
  have hSeparates :
      CnfClassifierSeparatesPolynomialCandidates model classifier :=
    cnfClassifierSeparates_of_sameDomainSeparator model hSameDomain
  exact Exists.intro classifier
    (And.intro hSeparates (hIndependent classifier hSeparates))
```

```
theorem cnfBareNegativeBranch_requires_independentSeparatorDiscriminator
    {Act Object : Type}
    {R : MinimalConditionsForAdmissibleConstruction.ConstructionRegime Act Object}
    {model : CnfEncodedCandidateModel}
```

```
    (hIndependent :
      CnfSeparatingClassifierIsIndependentSameDomain model)
    (hBare : CnfSATBareNegativeBranch R model) :
    CnfSATIndependentSeparatorEndpointStatusDiscriminator model :=
  cnfSATImageSeparatorBranch_requires_independentSeparatorDiscriminator
    (R := R)
    hIndependent
    (cnfBareNegativeBranch_forces_imageSeparatorBranch hBare)

theorem cnfSATImageSeparatorBranch_impossible_of_noIndependentDiscriminator
    {Act Object : Type}
    {R : MinimalConditionsForAdmissibleConstruction.ConstructionRegime Act Object}
    {model : CnfEncodedCandidateModel}
    (hNoIndependent : CnfNoIndependentSeparatingClassifier model)
    (hIndependent :
      CnfSeparatingClassifierIsIndependentSameDomain model) :
    Not (CnfSATImageSeparatorBranch R model) := by
  intro hSep
  exact Exists.elim
    (cnfSATImageSeparatorBranch_requires_independentSeparatorDiscriminator
      (R := R)
      hIndependent hSep)
    (fun classifier hClassifier =>
      hNoIndependent classifier hClassifier.1 hClassifier.2)
```

```
theorem cnfBareNegativeBranch_impossible_of_noIndependentDiscriminator
    {Act Object : Type}
    {R : MinimalConditionsForAdmissibleConstruction.ConstructionRegime Act Object}
    {model : CnfEncodedCandidateModel}
    (hNoIndependent : CnfNoIndependentSeparatingClassifier model)
    (hIndependent :
      CnfSeparatingClassifierIsIndependentSameDomain model) :
    Not (CnfSATBareNegativeBranch R model) := by
  intro hBare
  exact
    (cnfSATImageSeparatorBranch_impossible_of_noIndependentDiscriminator
      (R := R)
      hNoIndependent hIndependent)
      (cnfBareNegativeBranch_forces_imageSeparatorBranch hBare)

theorem cnfNotPositiveEndpoint_impossible_of_noIndependentDiscriminator
    {Act Object : Type}
    {R : MinimalConditionsForAdmissibleConstruction.ConstructionRegime Act Object}
    {model : CnfEncodedCandidateModel}
    (hNoIndependent : CnfNoIndependentSeparatingClassifier model)
    (hIndependent :
      CnfSeparatingClassifierIsIndependentSameDomain model)
    (hNotPositive : Not CnfPositiveEndpoint) :
    False :=
  (cnfBareNegativeBranch_impossible_of_noIndependentDiscriminator
    (R := R)
    (model := model)
    hNoIndependent hIndependent)
    hNotPositive
```

```
theorem cnfSATInPolyTime_of_noIndependentDiscriminator
    {Act Object : Type}
    {R : MinimalConditionsForAdmissibleConstruction.ConstructionRegime Act Object}
    {model : CnfEncodedCandidateModel}
    (hNoIndependent : CnfNoIndependentSeparatingClassifier model)
    (hIndependent :
      CnfSeparatingClassifierIsIndependentSameDomain model) :
    CnfSATInPolyTime :=
  cnfPositiveEndpoint_of_noIndependentDiscriminator
    (R := R)
    (model := model)
    hNoIndependent hIndependent
```

```
theorem cnfSATInPolyTime_of_context_noIndependentDiscriminator
    {Act Object : Type}
    {R : MinimalConditionsForAdmissibleConstruction.ConstructionRegime Act Object}
    {model : CnfEncodedCandidateModel}
    (_hCtx : CnfSATClayEndpointImageContext R model)
    (_hFixed : CnfSATFixedEndpointDomain R)
    (_hModel : CnfSATContextBoundCNFModel R model)
    (hNoIndependent : CnfNoIndependentSeparatingClassifier model)
    (hIndependent :
      CnfSeparatingClassifierIsIndependentSameDomain model) :
    CnfSATInPolyTime :=
  cnfSATInPolyTime_of_noIndependentDiscriminator
    (R := R)
    (model := model)
    hNoIndependent hIndependent
```

```
def CnfSATBookkeepingNegativeOnly
    {Act Object : Type}
    (_R : MinimalConditionsForAdmissibleConstruction.ConstructionRegime Act Object)
    (_model : CnfEncodedCandidateModel) : Prop :=
  Prop

def CnfSATCarrierShift
    {Act Object : Type}
    (_R : MinimalConditionsForAdmissibleConstruction.ConstructionRegime Act Object)
    (_model : CnfEncodedCandidateModel) : Prop :=
  Prop

def CnfSATOfficialNegativeEndpointUse
    {Act Object : Type}
    (_R : MinimalConditionsForAdmissibleConstruction.ConstructionRegime Act Object)
    (_model : CnfEncodedCandidateModel) : Prop :=
  CnfSATBareNegativeBranch _R _model

def CnfSATNegativeOccupationExhaustion
    {Act Object : Type}
    (_R : MinimalConditionsForAdmissibleConstruction.ConstructionRegime Act Object)
    (_model : CnfEncodedCandidateModel) : Prop :=
  CnfSATImageSeparatorBranch _R _model \/
  CnfOrdinaryNegativeAlternative _R _model \/
  CnfSATBookkeepingNegativeOnly _R _model \/
  CnfSATCarrierShift _R _model
```

```
theorem cnfSATNegativeOccupation_nonoptional
    {Act Object : Type}
    {R : MinimalConditionsForAdmissibleConstruction.ConstructionRegime Act Object}
    {model : CnfEncodedCandidateModel}
    (hOfficial : CnfSATOfficialNegativeEndpointUse R model)
    (hFixed : CnfSATFixedEndpointDomain R)
    (hModel : CnfSATContextBoundCNFModel R model)
    (hAMetric : AMetricBoundary R) :
    CnfSATImageSeparatorBranch R model :=
  -- formal proof in SATOperatorProofQueue.lean
```

```
inductive CnfSATEndpointStatus where
  | positive
  | separator

def CnfSATGovernedEndpointUse
    {Act Object : Type}
    (_R : MinimalConditionsForAdmissibleConstruction.ConstructionRegime Act Object)
    (_model : CnfEncodedCandidateModel) : Prop :=
  Prop

theorem cnfSATNegativeGovernedEndpointUse_has_separatorStatus
```

```
    {Act Object : Type}
    {R : MinimalConditionsForAdmissibleConstruction.ConstructionRegime Act Object}
    {model : CnfEncodedCandidateModel}
    (hGov : CnfSATGovernedEndpointUse R model)
    (hNotPositive : Not CnfPositiveEndpoint) :
    CnfSATImageSeparatorEndpointUse R model :=
  -- formal proof in SATOperatorProofQueue.lean

theorem cnfSATImageSeparatorBranch_of_negativeGovernedEndpointUse
    {Act Object : Type}
    {R : MinimalConditionsForAdmissibleConstruction.ConstructionRegime Act Object}
    {model : CnfEncodedCandidateModel}
    (hGov : CnfSATGovernedEndpointUse R model)
    (hNotPositive : Not CnfPositiveEndpoint) :
    CnfSATImageSeparatorBranch R model :=
  -- formal proof in SATOperatorProofQueue.lean
```

A.5 Focused axiom-output table

The focused audit files in the release issue `#print axioms` commands for the SAT endpoint bridge anchors below. The release records the following focused axiom surface for these proof-spine theorems: only the standard Lean/classical/mathlib base dependencies `propext`, `Classical.choice`, and `Quot.sound` appear. No project-specific axiom, `sorry`, `admit`, or `unsafe` declaration is used by these anchors.

Lean theorem	Focused axiom output
`cnfBareNegativeBranch forces_imageSeparatorBranch`	`[propext, Classical.choice, Quot.sound]`
`cnfImageSeparatorBranch of_not_positiveEndpoint`	`[propext, Classical.choice, Quot.sound]`
`cnfSATImageSeparatorBranch endpointUse`	`[propext, Classical.choice, Quot.sound]`
`cnfSATImageSeparatorBranch requires_independentSeparatorDiscriminator`	`[propext, Classical.choice, Quot.sound]`
`cnfSATImageSeparatorBranch impossible_of_noIndependentDiscriminator`	`[propext, Classical.choice, Quot.sound]`
`cnfBareNegativeBranch requires_independentSeparatorDiscriminator`	`[propext, Classical.choice, Quot.sound]`
`cnfBareNegativeBranch impossible_of_noIndependentDiscriminator`	`[propext, Classical.choice, Quot.sound]`
`cnfNotPositiveEndpoint impossible_of_noIndependentDiscriminator`	`[propext, Classical.choice, Quot.sound]`
`cnfSATInPolyTime of_context_noIndependentDiscriminator`	`[propext, Classical.choice, Quot.sound]`
`cnfSATNegativeOccupation exhaustion`	`[propext, Classical.choice, Quot.sound]`
`cnfSATNegativeOccupation nonoptional`	`[propext, Classical.choice, Quot.sound]`
`cnfSATGovernedEndpointUse bivalent`	`[propext, Classical.choice, Quot.sound]`
`cnfSATNegativeGovernedEndpointUse has_separatorStatus`	`[propext, Classical.choice, Quot.sound]`
`cnfSATImageSeparatorBranch of_negativeGovernedEndpointUse`	`[propext, Classical.choice, Quot.sound]`

A.6 Audit scope and boundary

The Lean archive formalizes the SAT-operator bridge and negative-branch closeout used in the manuscript proof spine. The current SAT-operator proof queue also includes negative-occupation-exhaustion anchors and endpoint-status-bivalence anchors, corresponding to commits `282d4ad` and `a938648` in the public Lean repository. It does not attempt to formalize the external Cook–Levin/Karp

theorem inside Lean, and it is not the source of the manuscript proof. Its support role is local and reproducibility-oriented: it checks that the formal SAT-side branch chain used by the manuscript has the advertised Lean anchors and that the audit surface is free of live project-level placeholders.

B Standard CNF-SAT to P=NP Correspondence

The manuscript proof derives $\mathsf{CnfSATInPolyTime}$. Here "bounded-CNF" means finitely encoded CNF formulas of arbitrary instance size over a finite Boolean alphabet; it does not mean a fixed-width, fixed-size, bounded-variable, fixed-clause, bounded-depth, promise-restricted, parameterized, or otherwise tractable restricted CNF fragment. The equality $\mathsf{P} = \mathsf{NP}$ then uses the standard Cook–Levin/Karp endpoint correspondence: arbitrary finite CNF-SAT is NP-complete under polynomial-time many-one reductions; every $L \in \mathsf{NP}$ reduces to CNF-SAT; composition of a polynomial reduction with a polynomial SAT decider gives a polynomial decider for L; and $\mathsf{P} \subseteq \mathsf{NP}$. Hence $\mathsf{NP} \subseteq \mathsf{P}$ and $\mathsf{P} = \mathsf{NP}$. This correspondence is standard Cook–Levin/Karp background. It is not the proof of separator exclusion; AASC supplies that proof.

C Reference Integrity Appendix

The proof ladder is ordered so that each theorem cites prior results only. The final endpoint theorem cites the context definition, positive direct-occupation theorem, model binding, the branch-local SAT image-separator bridge, raw complementarity, the raw-negative-to-image bridge, image-separator exclusion, and the standard CNF-SAT completeness bridge. No theorem cites itself.

D Notation Ledger

Symbol	Meaning	Layer
$\mathcal{C}_{\mathsf{SAT}}(R, m)$	fixed SAT endpoint carrier and branch-slot context	context
$\mathsf{Pos}_{\mathsf{raw}}$	$\mathsf{CnfSATInPolyTime}$	raw positive
$\mathsf{Sep}_{\mathsf{bare}}$	raw complement branch $\neg\mathsf{Pos}_{\mathsf{raw}}$	bare negative
$\mathsf{Sep}_{\mathsf{img}}$	formal SAT image-separator branch $\mathsf{CnfSATImageSeparatorBranch}(R, \mathit{model})$ induced by the raw negative branch under the lower-bound/no-positive bridge	image separator
$\mathsf{EndpointUse}$	branch B offered as theorem-bearing endpoint use	endpoint act
$\mathsf{OfficialEndpoint}$ $\mathsf{Resolution}$	ordinary theoremhood of $\neg\mathsf{Pos}_{\mathsf{raw}}$ offered as the official negative endpoint resolution	endpoint act
$\mathsf{EndpointDisc}$	endpoint-status discriminator attached to branch B	consequence layer
$\mathsf{IndependentDisc}$	independent same-domain governance work by C	forbidden case
$\mathcal{P}_{\mathsf{dir}}$	direct constructive occupation of the positive CNF-SAT endpoint	SAT asymmetry
$\mathcal{D}_{\mathsf{sep}}$	independent same-domain endpoint-status discriminator induced by the SAT image-separator branch	SAT asymmetry
$\mathcal{I}_{\mathsf{sep}}$	branch-local SAT independence bridge invoked by $\mathsf{Sep}_{\mathsf{img}}$ on the context-bound CNF model	SAT asymmetry

$\mathcal{N}_{\mathsf{sep}}$	kernel-forced no-independent-separator-classifier closure triggered by image-separator endpoint use	consequence layer
OfficialNegUse	official negative endpoint use on the context-bound finite CNF-SAT model	SAT negative occupation
NegOccExhaust	four-way exhaustion of official negative occupation alternatives	SAT negative occupation
GovUse	governed endpoint use on the finite CNF-SAT endpoint carrier	endpoint-status bivalence
EndpointStatus	two-status Lean surface: positive or separator	endpoint-status bivalence
A^{+}	kernel-forced consequence package for endpoint use	consequence layer
$\mathsf{A}_{\mathsf{sep}}$	role-specific separator status factor, downstream only	classification
$\mathsf{A}_{\mathsf{pos}}$	positive report status factor, downstream only	classification
$\mathsf{Pos}_{\mathsf{Clay}}$	$\mathsf{Pos}_{\mathsf{raw}} \wedge \mathsf{A}_{\mathsf{pos}}$	positive Clay mathematical endpoint

References

[1] S. Cook. *The P versus NP Problem*. Clay Mathematics Institute Millennium Problem description.

[2] A. J. Maley. *Non-Degenerate Construction and the Kernel of Admissibility*. 2026.

[3] A. J. Maley. *Claim Standing and Legitimacy*. 2026.

[4] A. J. Maley. *Anchor, Tensor, and Skin: A Fixed-Domain Decomposition Theorem for Admissible Description*. 2026.

[5] S. A. Cook. The complexity of theorem-proving procedures. In *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, pp. 151–158, 1971.

[6] L. A. Levin. Universal search problems. *Problemy Peredachi Informatsii*, 9(3):115–116, 1973. English translation in *Problems of Information Transmission*, 9(3):265–266, 1973.

[7] R. M. Karp. Reducibility among combinatorial problems. In R. E. Miller, J. W. Thatcher, and J. D. Bohlinger, editors, *Complexity of Computer Computations*, pp. 85–103. Plenum Press, 1972.

[8] S. Arora and B. Barak. *Computational Complexity: A Modern Approach*. Cambridge University Press, 2009.

[9] A. J. Maley. *AASC P vs NP SAT Endpoint Lean Audit*, version 1.0.0. Zenodo, 2026. DOI: `https://doi.org/10.5281/zenodo.20593794`.

[10] A. J. Maley. *AASC-PvsNP-SAT-Endpoint-Lean-Audit*. GitHub repository, 2026. `https://github.com/somamaley-ux/AASC-PvsNP-SAT-Endpoint-Lean-Audit`.